

重庆大学本科学生毕业论文（设计）

基于大模型和区块链的加密货币模拟交易所
的设计与实现



学 生：胡力文

学 号：20222053

指导教师：桑军

专 业：软件工程

重庆大学大数据与软件学院

2026 年 6 月

Undergraduate Thesis (Design) of Chongqing University

**Design and Implementation of a Crypto
Simulated Exchange Integrating LLMs and
Blockchain**



By

HU Liwen

Supervised by

Lect. SANG Jun

Software Engineering

School of Big Data & Software Engineering

Chongqing University

June, 202

摘 要

随着大语言模型的发展，Agent 在金融行业中的占比越来越高。虽然 Agent 可以在交易中发挥其处理信息和逻辑推理的强大能力，但由于安全风险，几乎没有团队会使用 Agent 直接操作线上交易。本系统设计并实现了一个集成多 Agent 引擎和测试区块链的模拟交易所，旨在测试不同提示词、不同基础模型 Agent 的交易能力，同时利用 AI 能力为用户提供股票交易建议。

本系统分为三个核心模块。

第一是模拟交易所，接入 Alpha Vantage 数据接口获取金融数据，使用 SpringBoot 与 Vue 3 实现前后端分离的交易平台，买卖操作仅在本本地 DB 执行，实现了与真实市场的隔离。

第二是并发 Agent 决策层，基于 LangChain4j 框架，注册了三位 AI 交易员，它们搭载着不同厂商的大语言模型，同时配置了保守、适中和激进三种风险偏好的提示词模板，Agent 在读取金融数据后，做出决策，调用工具自行执行交易。

第三是区块链层，笔者在 Ganache 测试网中部署了三个同样的 ERC-20 代币合约，由系统账户提供交易 Token 所需要的流动性，并使用线性冲击公式动态定价，实现了链上交易功能。

本系统主要实现了股票辅助决策与斗兽场博弈两部分功能，其一是 Agent 根据真实股票数据进行决策并执行买卖操作，其二为配置 AI 在测试链上进行循环的交易博弈。

本系统为大语言模型在金融场景中的行为研究提供了一个可编程、可复现的工程基础。

关键词：多 Agent 决策；金融交易模拟；区块链交易；LangChain4j

ABSTRACT

With the development of Large Language Models (LLMs), AI Agents have become increasingly prominent in the financial sector. Although Agents exhibit sophisticated information processing and logical reasoning capabilities, inherent security risks have deterred their direct application in live online trading. This system designs and implements a simulated exchange platform that integrates a multi-Agent engine and a test blockchain, aiming to evaluate the trading performance of Agents across various prompt templates and foundational models, while simultaneously providing users with AI-driven stock trading recommendations.

The system architecture is comprised of three core modules: first, a simulated exchange module that acquires financial data via the Alpha Vantage API and employs a SpringBoot and Vue 3 decoupled architecture, ensuring market isolation by confining trade executions to a local database; second, a concurrent Agent decision layer based on the LangChain4j framework, which incorporates three AI traders powered by heterogeneous LLMs and configured with conservative, moderate, and aggressive risk-preference prompts to perform autonomous decision-making and tool invocation; and third, a blockchain layer where three identical ERC-20 token contracts are deployed on a Ganache test network, with liquidity provided by system accounts and dynamic pricing governed by a linear impact formula to facilitate on-chain transactions.

The system primarily supports two functional scenarios: AI-assisted stock decision-making based on real-market data and a "Colosseum" game for iterative trading competition on the test chain. Ultimately, this work provides a programmable and reproducible engineering foundation for the behavioral research of Large Language Models within financial scenarios.

Key words: Multi- Agent Decision- Making; Financial Trading Simulation; Blockchain Trading; LangChain

目 录

摘 要	I
ABSTRACT	III
1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 大语言模型在金融领域的应用	2
1.2.2 模拟区块链的交易平台	2
1.2.3 模拟交易所的工程积累	3
1.3 主要研究内容	3
2 相关理论与关键技术	5
2.1 前后端框架	5
2.1.1 SpringBoot 框架	5
2.1.2 Vue 3 框架	5
2.2 Alpha Vantage 金融数据 API	5
2.3 LangChain4j 大模型调用框架	6
2.3.1 @Tool 注解	6
2.3.2 AiServices 代理机制	6
2.4 区块链与智能合约相关技术	6
2.4.1 以太坊与 ERC-20 代币标准	6
2.4.2 Web3j 交互库	7
2.4.3 Ganache 测试网络	7
3 模拟交易所系统需求分析	9
3.1 系统使用场景	9
3.2 功能性需求	9
3.2.1 模拟交易所基础功能	9
3.2.2 AI 辅助交易功能	10
3.2.3 区块链斗兽场功能	11
3.2.4 用例图	12
3.3 非功能性需求	13
3.3.1 数据一致性	13
3.3.2 多线程并发	13

3.3.3 外部 API 调用容错	13
3.3.4 前端响应与展示性能	13
3.3.5 系统可观测性	13
4 模拟交易所系统设计	15
4.1 系统架构设计	15
4.2 功能模块设计	16
4.2.1 行情数据模块	16
4.2.2 模拟交易模块	17
4.2.3 AI 决策模块	17
4.3 数据库表设计	20
4.3.1 E-R 图	21
4.3.2 股票行情相关表	22
4.3.3 AI 辅助交易相关表	23
4.3.4 斗兽场相关表	24
4.4 本章小结	27
5 模拟交易所系统实现	29
5.1 行情数据模块实现	29
5.1.1 股票数据查询与缓存	29
5.1.2 股票技术指标的计算	29
5.2 AI 决策模块实现	30
5.2.1 模型配置与加载	30
5.2.2 提示词模板管理	30
5.2.3 单次决策的无状态设计	32
5.2.4 线程池配置	32
5.3 模拟交易模块实现	33
5.3.1 买卖工具方法	33
5.3.2 AI 交易员初始化	34
5.4 区块链斗兽场模块实现	34
5.4.1 角色配置与启动	34
5.4.2 竞争排序与轮转执行	34
5.4.3 代币兑换与价格更新	36
5.4.4 链上交互与私钥管理	37
5.5 前端与后端的交互实现	37
5.5.1 实时数据推送	37

5.5.2 页面组件结构	38
5.5.3 配置页面的实现	38
5.6 异常处理与日志	38
6 模拟交易所系统测试与分析	39
6.1 测试环境	39
6.2 股票辅助交易功能测试	39
6.2.1 行情数据展示	39
6.2.2 AI 单次决策过程	40
6.2.3 资产曲线展示	42
6.2.4 股市行情分析功能验证	43
6.2.5 交易风险评估功能验证	44
6.3 区块链斗兽场功能测试	45
6.3.1 斗兽场配置与启动	45
6.3.2 博弈过程与资产变化	45
6.3.3 决策日志与链上记录	46
6.4 本章小结	47
7 总结与展望	49
7.1 本文工作总结	49
7.2 不足与改进方向	49
致 谢	53
原创性声明和使用授权书	

1 绪论

1.1 研究背景与意义

2024 年下半年，彭博社的一则新闻报道阐述：在三个月内，一些对冲基金企业的 GPT-4 调用量提升了接近四倍，以 Balyasny Asset Management、Two Sigma Investments 为代表的基金正竞相将 ChatGPT 应用于日常研究与投资流程^[1]；并且桥水基金（Bridgewater）宣布了成立规模超过 20 亿美元的机器学习基金，其模型来源包括 OpenAI、Anthropic 等^[2]。这些激增的 token 数并不是简单地用来编写周报，而是直接用于生成交易策略。由此可见，在金融分析领域，大语言模型正在从边缘工具逐渐向着核心的交易决策环节靠拢。

摩根大通在 2023 年启动的 indexGPT 项目更是直接印证了这一趋势^[3]，从其招聘要求可以推测，该项目的目标是搭建一个能够理解市场数据并独立执行操作的智能系统。要构建从信息到执行的端到端交易系统，存在着许多阻碍，其中最绕不开的就是安全问题。2023 年 Euler Finance 作为备受信赖的去中心化借贷协议，仅仅因为智能合约漏洞，就遭到攻击，损失了高达 1.97 亿美金^[4]，成为当年最大的 DeFi 安全事件。由此可见，即使是一个小小的安全漏洞，就能导致如此灾难性的后果，让 AI 这类未经充分验证的系统下放真实市场，必然会带来更大的安全隐患。十分追求稳定性的金融团队，不会接受 Agent 的不确定性，这就导致了 AI 技术难以在金融领域推广。

这两重障碍牵引出了一个明确的需求。一个模拟的实验环境，既能向模型开放完整交易能力，又能将风险边界限制在系统内部，本课题的落脚点即在此。

本系统由三层架构组成：第一层为模拟交易所，它提供查询行情与买卖交易的功能。所有交易仅在本地数据库中记账，不触达真实市场。第二层是多 Agent 决策层，基于 LangChain4j 框架驱动三个模型，它们可以持有不同风险偏好。这些模型读取相同的市场数据，自主决定买卖或观望，构造指令由第一层执行。

第三层是本地的测试区块链，笔者在测试链上部署了最简约的 ERC-20 代币合约。为了让 AI 能够在风险可控的环境下博弈，笔者专门设计了一个名为“斗兽场”的端到端试验环境。在这个环境里，三位 AI 交易员持有一定初始资金，在去中心化的代币池中不断循环、相互交易。所有代币的价值由链下公式根据供需动态决定，既避免了真实 Gasfee 对策略执行的干扰，又能观察 AI 在零和博弈下的决策思路。

在 AI 时代，本系统可以为金融工作者提供一个没有风险的实践平台；同时，也能够体现 AI 在相同市场、不同风险偏好下，决策和收益的差异，这对投资者用

户可以起到一定的参考作用。

1.2 国内外研究现状

1.2.1 大语言模型在金融领域的应用

AI 在金融垂直领域的探索，早期研究主要是通用模型的指令微调。Yang 等人在 2023 年提出的 FinGPT 框架，就验证了轻量化模型（如 LLaMA、ChatGLM）用金融数据进行微调的工程可行性^[5]。这类工作表明了了在 prompt 中注入金融情感分析、财报数据和社交媒体舆情，小模型也能具备敏锐的市场感知力。

随着对大模型研究的深入，金融领域大模型也实现了从单轮对话到长期记忆的演进。2023 年推出的 TradingGPT 引入了分层记忆模块(Layered Memory Module)，使 Agent 在外汇与股票交易任务中能够吸取历史的经验，其量化表现显著优于传统的 Baseline 模型^[6]。

虽然上述学术成果在模拟评测中表现优异，是十分有价值的工作基础，但其共同的局限性在于其“交易行为”始终停留在理论模拟和离线的测试阶段。而且现有的金融 Agent 大多被当作“投资顾问”而不是独立的交易员，都未能打通“信息-推理-执行”的端到端闭环。在缺乏适配 API 的现状下，部分研究试图让 Agent 通过操作浏览器或者编写 Selenium 自动化脚本来执行交易。这种模式看起来可行，但在并发量高、可靠性要求极高的金融场景下，会容易因为网页变动或脚本接口变动导致任务执行失败。因此，构建一套体系化、标准化的 Agent 工具链，是金融 Agent 走向实战的重要前置条件。

1.2.2 模拟区块链的交易平台

日渐完善的区块链技术为去中心化交易所的模拟研究提供了重要的底层环境。

2022 年发表于 IEEE Xplore 的论文《SlipSwap》尤其具有代表性^[7]，该工作的试验环境直接由以太坊测试工具 Ganache 一键搭建，同时在自动化做市商公式的基础上编写智能合约，数据源选用 Binance 的历史回放。该工作精细地复现了链上的流动性枯竭、三明治攻击以及滑点损耗等区块链金融现象。

在另一类研究中，研究者意在通过对极端交易环境的模拟，评估智能合约的抗风险能力。例如《Agent-Based Modeling of Blockchain Decentralized Financial Protocols》一文^[8]。该工作是以 Uniswap 去中心化交易所为工程底座，构建了抢跑者（Front-runner）与套利者（Arbitrageurs）等不同角色的主体，重点研究了在真实市场压力以及恶意套利环境下，点对交易滑落和协议利润分配的实际影响。

但是，上述的研究都存在一个共同缺陷，它们执行交易的主体存在局限。比如，《SlipSwap》中使用固定的脚本回放 Binance 的历史数据。《Agent-Based Modeling of Blockchain Decentralized Financial Protocols》中的 Front-runner 是利用触发式的

脚本监听大额交易，地以更高的 Gas Fee 发起新交易；Arbitrageurs 则是一个确定性的数学优化算法，监听流动池中的价格，满足套利条件则触发买入或卖出指令。这些工作的行为主体都依赖于硬编码的脚本或者算法，它们的交易逻辑是死板的线性的，它们无法像真正的交易员一样应对复杂的市场情况，更不具备深层的推理决策能力。

但在另一方面可以发现，Hardhat、Ganache 等本地测试链工具降低了去中心化环境的部署门槛；成熟的去中心化交易所实现方案，为笔者模拟去中心化交易提供了重要的工程基础。

1.2.3 模拟交易所的工程积累

在金融工程中，模拟交易所是一项发展了数十年的成熟技术，但很少有工作将 AI 和模拟交易的工程基础结合起来。

从国外研究来看，学术界在金融 Agent 上进行了一定程度的探索，例如 2024 年提出的工作 StockAgent^[9]。该系统接入了多个大模型（如 GPT-3.5、Gemini），构建了多 Agent 金融交易模拟系统，在这个系统中 Agent 扮演投资者对市场外部事件做出交易决策。但是，这项工作也仅仅局限在离线数据的回放测试上，并没有接入真实的券商接口，缺少真实的标准化 API 设计，并且也未能将 Agent 交易领域扩展到区块链生态中。

在国内来看，从 2024 年开始，国内高校就出现了几篇涉及“AI 模拟交易”的毕业设计，他们使用的技术各不相同，但都没有实现现代工程的 Agent 概念，他们要么依赖固定的 LSTM 模型^[10]来单纯预测次日的涨跌幅度，要么仅仅将大模型用作前端的聊天客服组件，用户可以通过自然语言操作股票交易，但底层的交易系统依然是传统的工程代码。

在工程架构方面，Java 生态中 Langchain4j 这一新兴架构在金融领域中也出现得非常少，尤其是 Agent 并发调用 Tools 的复杂场景更是缺少公开的架构设计。

由此可见，这三条路线的交叉地带存在着不小的空白，本项目是将大语言模型 Agent 的推理和工具调用能力、模拟交易所系统性的 API 和测试区块链的去中心化技术合三为一的系统框架，这在工程中尚无实现案例。

1.3 主要研究内容

本课题的工作可以划分为三个模块。

模拟交易所平台的构建，前后端分离设计。前端 Vue 3 承担行情展示与 Agent 表现展示等功能。后端 SpringBoot 提供 RESTful 接口，买卖功能不接入真实交易系统，只操作 MySQL 资产表。买卖接口以工具形态封装。区块链和股票市场交易共用同一个买卖 API。

基于 LangChain4j 的多 Agent workflow。该流触发后，系统从数据库读取行情数

据，计算各支股票的技术指标，并在线程池中创建符合用户配置的 AI 交易员。本系统共接入了三种 AI 模型，其区别为底层模型不同（ERNIE 5.0^[11]、qwen-Max^[12]、GPT-4o-mini^[13]），并且笔者预设了三种不同的提示词，分别为保守、适中、激进三种风险取向。在单次模拟中将指标与持仓数据投喂给模型，模型在经过逻辑推理后决定观望或者调用买卖工具 API，最后将执行结果和决策理由落库，推送前端。

测试区块链与代币合约——“斗兽场”博弈环境。笔者在本地 Ganache 测试链上部署三种 ERC-20 代币合约，构建了一个封闭的多 Agent 博弈环境。用户配置角色组合与博弈轮数后，系统为每位 Agent 分配等额初始代币。每轮交易中，三位 Agent 依次进行代币兑换。代币价格由链下线性冲击公式根据供需动态更新，并且用“SET”作为一般等价货币用于折算 Agent 的资产总值。该测试链的 Gas 费为零，目的在于简化系统，并且消除交易成本对决策的干扰。

2 相关理论与关键技术

本章介绍系统开发过程中涉及的主要技术，包括前后端框架、行情数据 API、LLM 调用框架以及区块链相关组件。对于各项技术，笔者在此重点说明其基本概念和核心功能，本系统的具体实现方式将留在后续章节展开。

2.1 前后端框架

2.1.1 SpringBoot 框架

SpringBoot 是基于 Java 语言的后端开发框架，其设计目标是简化 Spring 应用的搭建和开发^[14]。与传统 Spring 框架相比，SpringBoot 通过自动配置和起步依赖两个核心环节，大幅减少了笔者在配置文件和依赖管理上的工作量，能够更专注于业务逻辑的编写。

SpringBoot 的核心特性包含内嵌的 Tomcat 服务器，应用可直接以 Java 程序的方式运行；提供大量起步依赖，保障了初始服务的可用性，同时工程师也可以在几行 Maven 代码内引入所需的组件；支持注解管理和注入 Bean 类等切面编程的能力。在本系统中，SpringBoot 主要用于构建 RESTful API、运行数据库事务以及调度定时任务。

2.1.2 Vue 3 框架

Vue 是一套用于构建前端界面的渐进式 JavaScript 框架^[15]。Vue3 在 Vue2 的基础上引入了组合式 API、更好的 TypeScript 支持和基于 Proxy 的响应式系统。

笔者通过组合式 API 能够将组件按照功能的逻辑组织，而可以不用依赖程序的生命周期，这提高了代码的复用性和可维护性。在性能上，Vue3 基于 Proxy 的响应式架构明显优于 Vue2 使用 Object.defineProperty 实现的方案，尤其是在数据量与日俱增的场景下，Vue3 的响应速度更具备优势。所以，本系统的前端界面基于 Vue 3 构建，配合 Element Plus 组件库和 ECharts 图表库完成交易所布局与股票数据可视化。

2.2 Alpha Vantage 金融数据 API

Alpha Vantage 是一个提供免费金融数据服务的 API 平台，由 Y Combinator 孵化^[16]。该 API 提供股票、外汇和加密货币等金融资产的价格信息，支持实时报价、历史 K 线以及近百种技术指标。

笔者调用 Alpha Vantage 的 API 为 RESTful 风格，调用方式为 HTTP GET 请求，基础 URL 为 <https://www.alphavantage.co/query>。请求参数主要包括 function（指定数据类型，如 TIME_SERIES_DAILY 表示日线数据）、symbol（股票代码）和 apikey（用户身份凭证），笔者阅读接入文档后，处理返回的 JSON 格式数据。

考虑到本系统并未商用，所以笔者选择了接入免费的 API，因其存在调用频率和日调用总量的限制，笔者还设计了一套数据缓存策略，来降低使用数据 API 的频率，这将在后文详细介绍。

2.3 LangChain4j 大模型调用框架

LangChain4j 是新兴的、结合 Java 生态的 Agent 开发框架^[17]，它填补了 Java-Spring 生态与 LLM 生态之间的适配缺口，它的核心目的是让开发者能够用符合 Java 习惯的方式在程序中集成 LLM 的推理能力。在该框架下，笔者就避免了冗杂的 HTTP 调用、JSON 解析和手动管理对话上下文等繁琐易错的细节。

LangChain4j 提供了一系列功能，其中与本系统最相关的两个概念是 @Tool 注解和 AiServices 代理。

2.3.1 @Tool 注解

Tool 注解可以将普通的 Java 方法标注为可供大模型调用的工具。框架在启动时会通过反射获取 Java 方法的元信息，包括方法名、参数列表和参数名称，并将方法信息转换为大模型可以理解的自然语言。调用大模型时，这些工具描述会随提示词一同发送，这时候，模型在进行推理后，判断是否调用某个工具，并在响应中返回调用指令。框架拦截调用信息（方法名、参数信息等）后，反射执行对应的 Java 方法，并将返回值反馈给模型以生成最终回复。

笔者在 Tool 注解中填入了交易所买卖的功能描述，大模型会依据该描述判断应该传入哪些参数，这将直接影响工具调用的结果，具体的使用如下

```
@Tool("买入指定股票。symbol 为股票代码，amount 为买入股数。")
public Boolean buy(Integer amount, String symbol);
```

2.3.2 AiServices 代理机制

AiServices 是 LangChain4j 提供的另一个核心组件，其作用是将大模型封装为动态代理对象。开发者只需定义一个方法接口，通过 AiServices.Builder 注入配置后的基础大模型、RAG 存储、和上文介绍的交易工具等，即可获得该接口的代理对象。

开发者调用 AiServices 代理方法时，框架会自动进行提示词构建、请求发送、响应解析以及工具的调用，最终返回大模型输出的结果。这整个复杂的流程对开发者而言，只是一次普通的方法调用，这显著提升了可读性，降低了代码复杂度。

2.4 区块链与智能合约相关技术

2.4.1 以太坊与 ERC-20 代币标准

以太坊是最常用的支持智能合约的公链，开发者可以在其链上部署去中心化的 DAPP。智能合约就是运行在以太坊虚拟机中的代码，Pos 共识机制保证其执行结果不被篡改。

而 ERC-20 是以太坊上最广泛使用的代币接口标准^[18], 遵循该标准的智能合约都要实现六个方法: `totalSupply` (返回总供应量)、`balanceOf` (查询地址余额)、`transfer` (转账)、`transferFrom` (授权转账)、`approve` (授权额度) 和 `allowance` (查询授权额度)。这六个接口实现了一个交易单位最基本的交易功能。

统一的接口让各种钱包、交易所和工具能够无缝兼容任何合约, 只要它符合 ERC-20 规范。本系统的测试代币也实现了 OpenZeppelin 提供的 ERC-20 标准^[19], 暴露给 AI 的买卖工具中, 就涉及到测试链上 ERC-20 合约的调用。

2.4.2 Web3j 交互库

Web3j 是一个用来让 Java 程序与以太坊节点进行交互的库^[20]。它封装了以太坊的 JSON-RPC 接口, 为笔者这样的 Java 开发者提供了符合 Java 逻辑的合约调用方式。

Web3j 的主要功能有: 连接以太坊节点 (HTTP 和 IPC)、管理账户与私钥、部署智能合约以及调用合约的读写函数等。它还提供了 `client` 工具和 `Maven` 插件, 因此笔者提供 Solidity 合约的 ABI 和字节码就可生成对应的 Java 封装类, 能够以面向对象的方式调用合约函数, 而且无需手动连接以太坊节点、构造 JSON-RPC 请求和执行字节码文件等繁琐操作, 极大地提升了笔者的开发效率。

2.4.3 Ganache 测试网络

Ganache 是以太坊开发工具套件 Truffle Suite 的一部分^[21], 该组件可以在本地快速搭建以太坊测试网络。Ganache 启动后会在本机运行一个完整的以太坊节点, 其图形界面允许开发者逐块查看链上发生的交易、查看账户余额变化以及调试合约执行。由于是本地私有链, Gas 费用可设为零, 这简化了系统的复杂度, 让 Agent 更专注于彼此间的推理博弈, 非常适合开发和测试阶段的快速迭代。

3 模拟交易所系统需求分析

本章的主题是，系统到底要做什么，不做什么。在动手写代码之前，先把功能和非功能的要求理清楚，给后续设计和实现一个明确的标准。笔者将先阐述系统的使用场景和角色，然后分三个模块列出功能性需求，最后探讨非功能性需求。

3.1 系统使用场景

本系统的使用角色主要是对 AI 交易行为感兴趣的研究员或学习者，并不涉及真实的交易，系统分共为两个独立的场景。

第一个场景是股票辅助交易：使用者选择 AI 模型和风险偏好（配置有三种语言模型，和三种风险偏好类型），触发任务后 AI 分析行情并执行买卖，使用者可以观察 AI 的决策逻辑和收益。

第二个场景是区块链斗兽场：使用者配置 AI 数量、偏好类型和博弈轮数等参数，AI 进行多轮博弈，前端展示 AI 的收益曲线和决策逻辑。

3.2 功能性需求

功能性需求分为三个部分：交易所基础建设、Agent workflow、区块链斗兽场。

3.2.1 模拟交易所基础功能

这部分提供一套供 AI 执行买卖的基础设施。它不涉及真实资金，也不对普通用户开放账户注册功能。图 3.1 是模拟交易所功能的用例图。

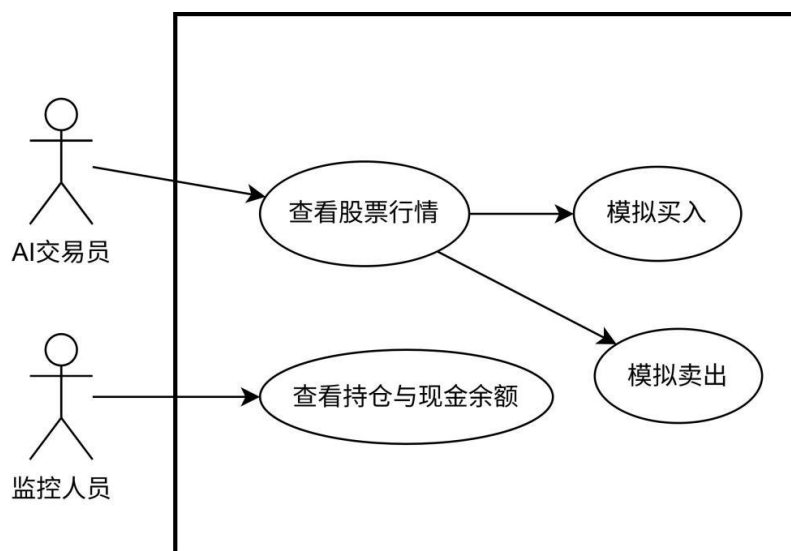


图 3.1 模拟交易所用例图

股票行情查看。系统需要展示若干只股票的基本信息和实时价格，数据来源

于 Alpha Vantage。前端以 k 线图形式显示股票代码、当前价格和涨跌幅。

模拟买入功能。AI 能够调用买入工具，输入股票代码和买入数量，返回执行结果。系统需要检查该股票是否存在、AI 账户中的现金是否足够支付，通过后流水落库，并更新持仓记录和现金余额。

模拟卖出功能。AI 能够调用卖出操作，传入股票代码和卖出数量，返回执行结果。系统需要检查 AI 持仓数量是否足够，确认后流水落库，并扣减持仓并增加现金余额。

持仓与现金读写。前端需要展示每位 AI 当前的持仓明细和现金余额，AI 自身在决策时也需要读写这些数据。

3.2.2 AI 辅助交易功能

这部分功能让使用者可以手动触发 AI 对真实股票行情的分析和操作，并感知到 AI 的决策理由。图 3.2 是 AI 辅助交易的用例图

模型与风险偏好选择。前端需要提供下拉菜单，让使用者从三个 AI 模型（ERNIE 5.0、通义千问 Max、GPT-4o-mini）中选择一个，再选择风险偏好（稳妥、适中、激进）。点击“AI 决策”按钮后，系统启动该 AI 的一次决策流程。

行情分析与决策。AI 被触发后，需要读取当前所有可交易股票的最新价格和近期技术指标，结合自身持仓情况，判断是否买入、卖出或者保持观望。如果决定操作，AI 需要调用提供的交易工具执行对应的买卖动作。

决策理由展示。无论 AI 最终是否交易，都需要输出一段文字说明它的判断依据，并且给出对当前股票和市场的分析。这段文字不需要保存，仅在前端的日志区域显示。例如“IBM 股价站上 20 日均线且 RSI 处于低位，因此买入 100 股”。

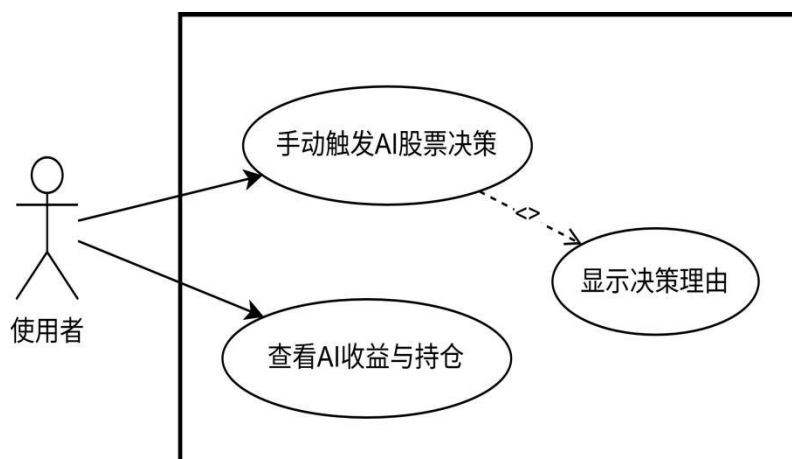


图 3.2 AI 辅助交易所用例图

收益展示。前端需要展示当前选中 AI 的累计收益率和资产净值，资产净值根

据现金加上持仓股票按市价折算的总和来计算；还需要展示 AI 的持股详情，持有某只股票多少股等。

3.2.3 区块链斗兽场功能

这部分是一个独立功能，和之前股票辅助交易不同，这里的交易对象是系统内部发行的三种 ERC-20 代币，交易形式是代币之间的兑换，不涉及现金。

角色配置与启动。使用者需要能够在页面上选择 AI 数量，为 AI 分别选择模型代号和风险偏好，输入想要运行的总轮数，并设置三种代币的初始数量。点击启动按钮后，系统开始运行。

竞争与依次执行。每一轮开始后，主线程需要先读取当前市场数据，为贴合真实的博弈场景，AI 需按照设计的竞争规则确定本轮的决策顺序。顺序确定后，三位 AI 依次执行各自的决策。第一位 AI 操作后市场价格可能发生变化，第二位 AI 将在新价格下做决策，第 n 位同理。单轮结束后，如果还没到设定的总轮数，就自动进入下一轮。

代币价格动态更新。三种代币 T1、T2、T3 的相对价格需要在每次 AI 执行兑换操作后自动更新，价格变化应反映供需关系，买入操作会推高该代币价格，卖出操作会拉低价格。

收益曲线展示。前端需要以折线图实时显示三位 AI 的资产总值变化。资产总值按照各 AI 持有的 T1、T2、T3 数量乘以当前价格再求和得到。曲线需要随着轮次推进自动更新。

决策日志与链上记录。前端需要列出每一轮每位 AI 的操作内容和决策理由。同时需要提供类似区块浏览器的功能，展示测试链上发生的转账记录，包括交易哈希、收发地址、代币种类和数量。

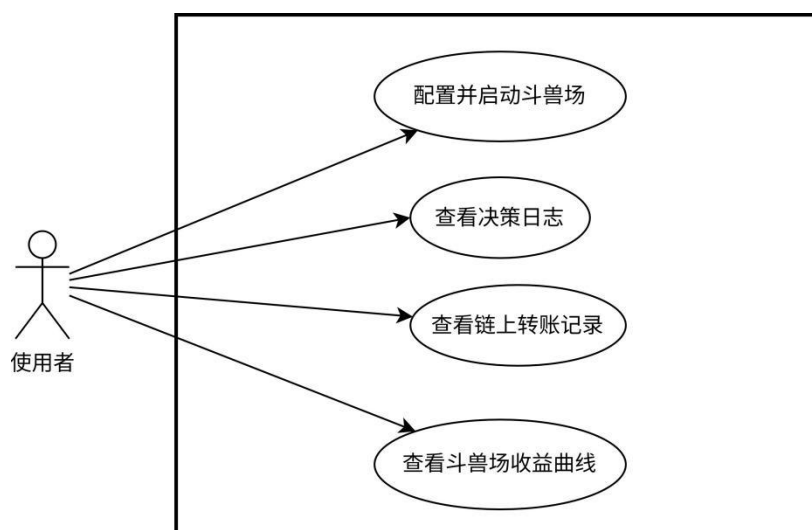


图 3.3 区块链斗兽场用例图

使用者权限限制。斗兽场启动后，使用者只能观看，不能手动干预 AI 的持仓，也不能暂停或修改运行参数，系统需要完整跑完预设轮数。

3.2.4 角色用例图

在明确了系统的使用场景和功能性需求之后，笔者需要从用户、Agent 的交互角度明确系统的功能范围，所以本节引入用例图对系统进行剖析。

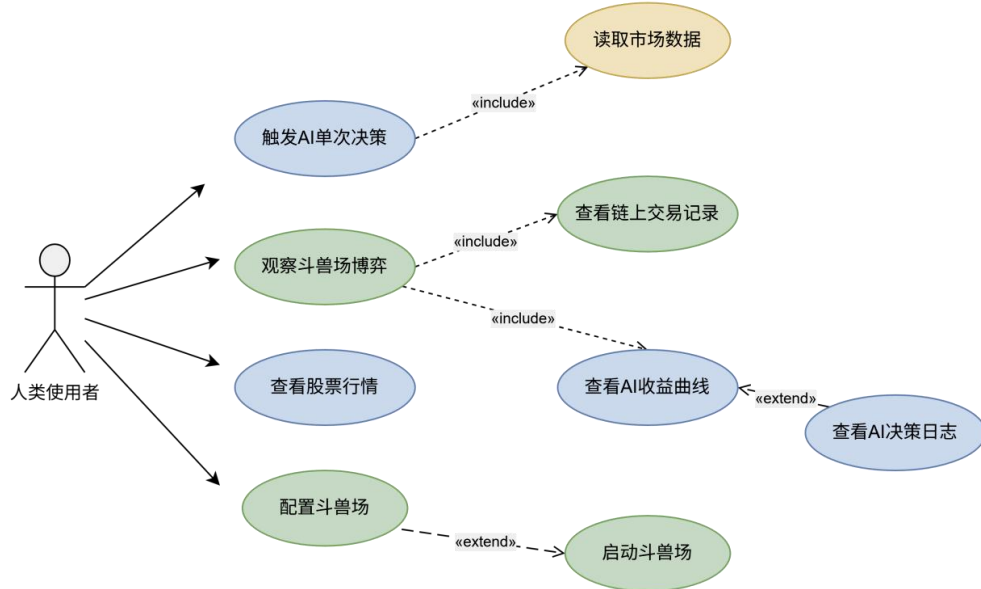


图 3.4 人类使用者用例图

“人类使用者”是本系统的主要参与者。如图 3.4 所示，人类使用者是系统的直接操控人，可以查看股票行情、触发 AI 决策以及查看 AI 日志与收益等；除此之外，人类操作者还可以使用“斗兽场”模块，能够配置 AI 对局设置、观察斗兽场博弈以及链上的交易记录。

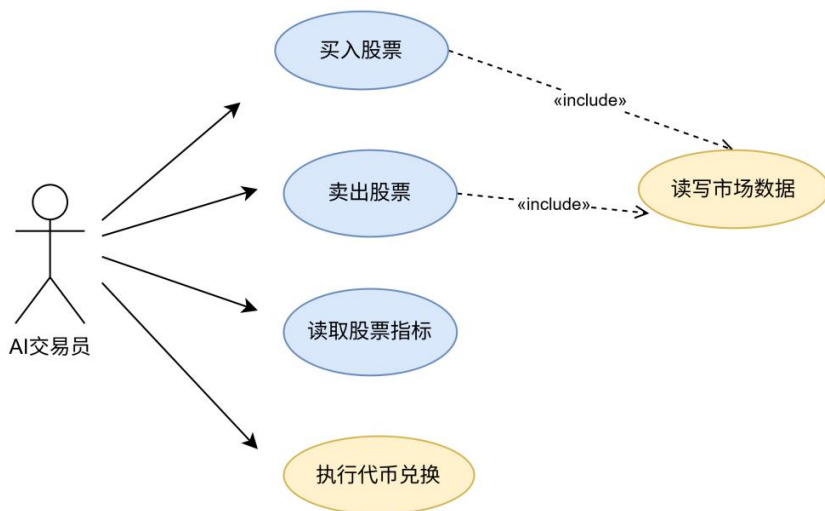


图 3.5 AI 交易员用例图

在本系统中，AI 交易员跟人类使用者一样，也是具备主观能动性的参与者，它在斗兽场和模拟交易所中都发挥着关键的作用，可以执行代币兑换和买卖股票操作，其中买卖操作包含了读写市场数据，如图 3.5 所示。

从用例关系看，“观察斗兽场博弈”包含了查看交易记录以及能够给用户带来参考价值的 AI 决策日志功能，并且人类用户触发 AI 决策，也能够同时读取市场数据。

3.3 非功能性需求

非功能性需求关注的是本系统在性能、可靠性、安全性等方面的要求。

3.3.1 数据一致性

股票买卖和代币兑换都会并发修改多张数据表。比如买入股票时，现金余额要减少、持仓要增加、交易流水要新增。系统必须保证这些更新操作要么全部成功，要么全部失败，不能出现一半成功一半失败导致数据对不上的情况，即需要满足事务的原子性。

3.3.2 多线程并发

斗兽场每轮需要多位 AI 竞争交易顺序，AI 辅助交易模块也可能同时收到多次手动触发。大模型 API 的调用是耗时的阻塞操作，如果放在主线程里执行会影响整个系统的性能，因此 AI 决策过程需要在独立的线程池中执行，线程池参数需要合理配置。

3.3.3 外部 API 调用容错

系统依赖 Alpha Vantage 和 LLM 两类外部 API。网络波动、API 配额耗尽、服务端故障等情况都可能导致调用失败，系统不能因为外部 API 出错就直接崩溃，所以需要有时限控制和重试机制，并在出错时向使用者给出明确提示。

3.3.4 前端响应与展示性能

斗兽场中的 AI 资产曲线会随着轮次增加积累大量的数据点，决策日志条目也会越来越多。前端渲染时需要注意性能，图表更新应采用增量方式只追加新数据，而不是每次都重新绘制整个图表。日志列表如果太长，也需要考虑分页或虚拟滚动。

3.3.5 系统可观测性

为了方便调试和事后分析，系统需要记录关键操作的日志，包括每次 AI 决策的输入数据、返回的原始内容、工具调用的参数和结果、链上交易哈希等。日志级别需要能够灵活调整，日常运行时用较粗略的级别，需要排查问题时再切换到详细级别。

4 模拟交易所系统设计

系统总体设计在需求分析的基础上进行，主要解决三个问题：系统采用什么样的架构、包含哪些功能模块、数据库需要新建哪些表来存数据。本章依次给出架构设计、模块划分和数据库表结构。

4.1 系统架构设计

本系统采用前后端分离的架构。前端使用 Vue 3 框架，运行在浏览器中，负责界面展示和用户交互。后端使用 SpringBoot 框架，运行在 Tomcat 服务器上，负责处理业务逻辑、操作数据库以及与外部系统通信。前后端之间通过 HTTP 协议传输 JSON 格式的数据。系统整体大致可以分为四层，如图 4.1 所示。

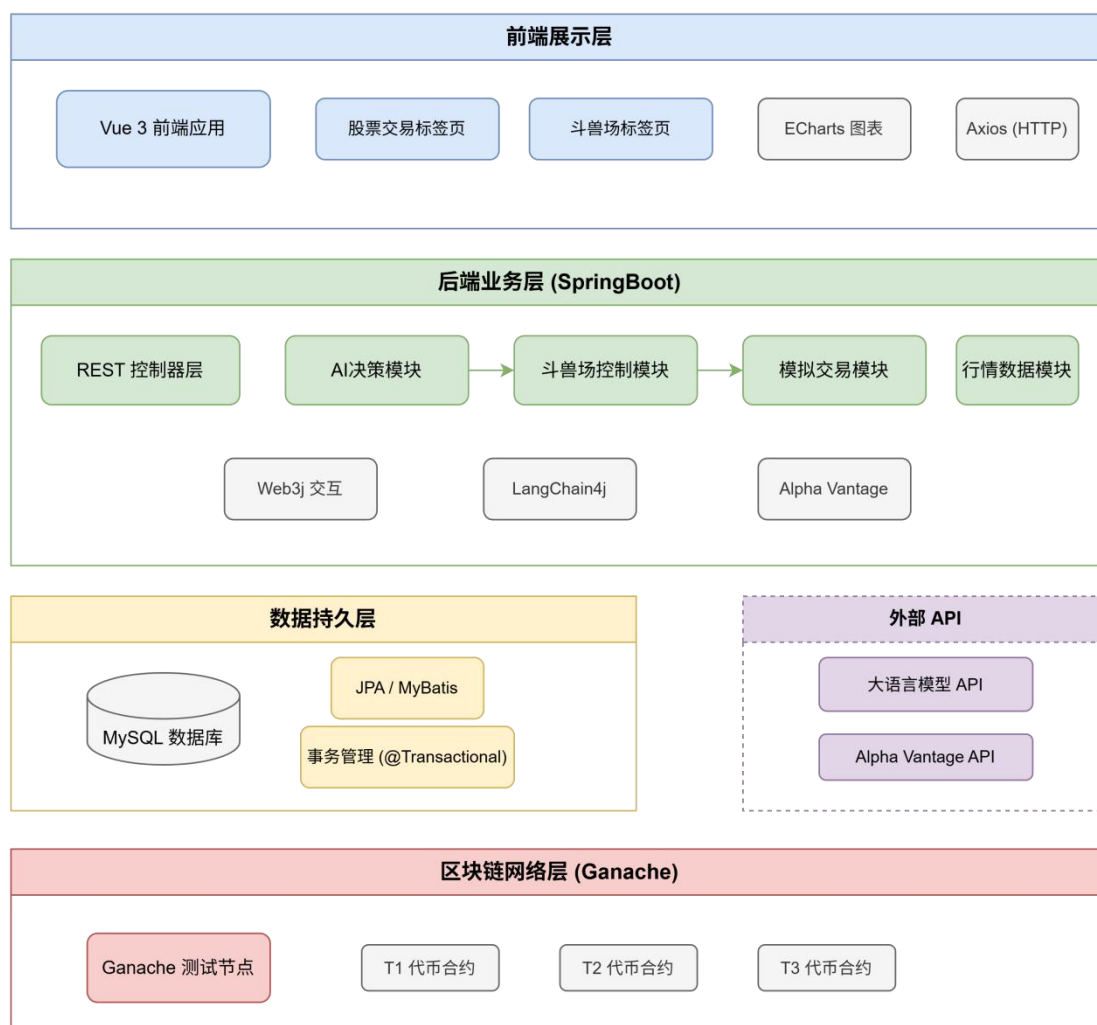


图 4.1 系统架构图

前端展示层，该项目的网页包含两个主要标签页，一个是股票交易页面，另一个是斗兽场页面。页面上有表格、图表、按钮和日志列表，用户通过它们查看数据、触发操作。

后端业务层，这一层是 SpringBoot 应用，接收前端发来的请求，执行对应的业务逻辑，然后返回结果。业务层内部又分为控制层、服务层和数据接口层。控制层负责接收请求和返回响应；服务层负责具体的业务处理，比如计算技术指标、调用大模型、执行买卖操作等；数据接口层负责读写持久层的数据

数据持久层。该层由 MySQL 存储中间件和 MyBatis 持久层框架实现，用来存储股票信息、价格历史、AI 持仓、交易记录、斗兽场配置等所有需要持久保存的数据。

区块链网络层。这一层是本地运行的 Ganache 测试链，链上部署了三种 ERC-20 代币合约。后端通过 Web3j 库连接 Ganache 节点，执行查询余额和转账等操作。

此外，系统还依赖两个外部 API：Alpha Vantage 提供股票行情数据，大语言模型 API 提供 AI 推理能力，后端通过 HTTP 和 JSON-RPC 调用这两个外部服务。

4.2 功能模块设计

按照功能需求的划分，本系统需要设计模块来实现模拟交易所功能、AI 辅助交易功能和区块链斗兽场功能，针对功能需求的三个场景，笔者设计了六个相辅相成的模块。

4.2.1 行情数据模块

行情数据模块，该模块是整个系统的数据基础设施，负责抽象并封装外部的行情数据源。包括外部 API 的数据接入、清洗，以及本地数据库的调用。通过提供约定的数据 Class（DAO），实现为上层业务模块屏蔽底层的数据源差异，保障了上层模块高效的查询性能和获取数据的快捷可读。

如图 4.2 所示，行情数据模块的数据查询功能采用懒加载策略，具体设计流程如下：

按需发起查询，项目启动时不会拉取行情接口的全量信息，仅在前端请求指定股票的 K 线图时才触发后端查询逻辑。

本地 DB 检索，后端接收到前端请求后，首先查询本地数据表，若存在对应的数据则直接读取并返回前端。

外部数据补充，若查询未命中本地 DB，则使用 RestTemplate 类调用 Alpha Vantage 数据接口。

数据解析落库，收到响应后使用 Jackson 静态类解析 JSONResponse，提取每日收盘价、最高价、最低价后，批量插入本地数据库表。

后续请求复用，后续相同 K 线数据的查询请求将直接命中本地 DB，这样就实

现了在有限的 API 使用限度内，节省网络与存储资源。

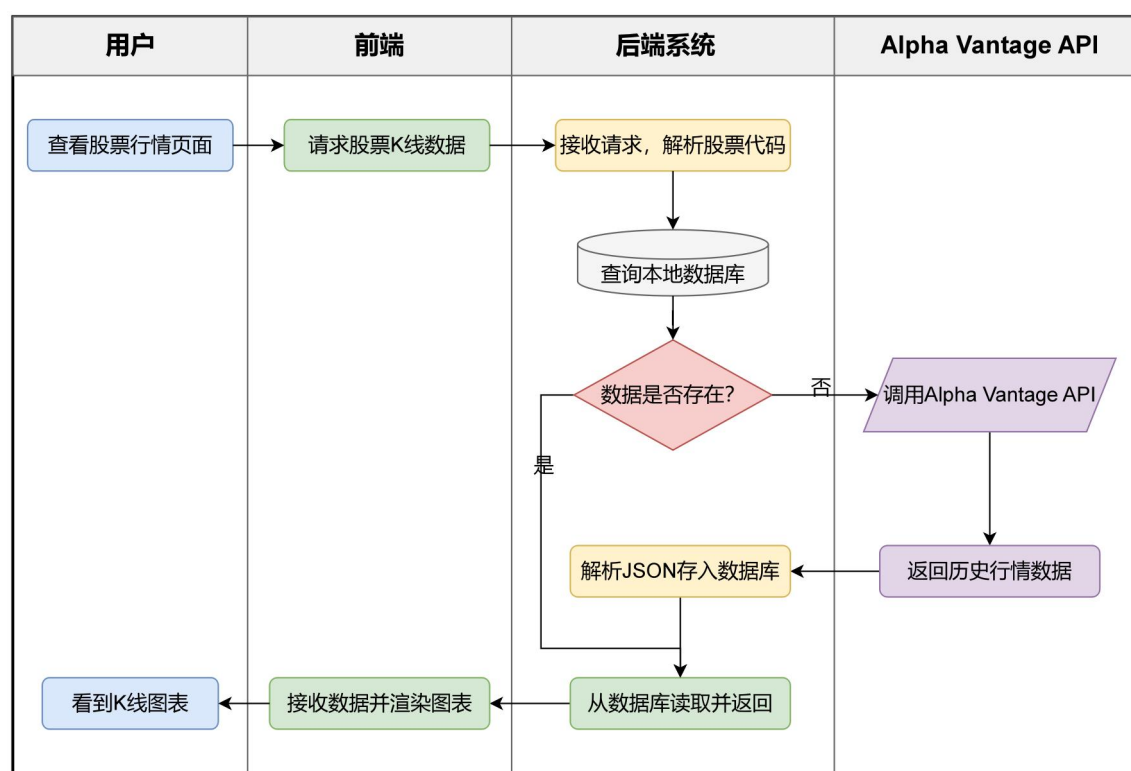


图 4.2 查询股票数据流程图

4.2.2 模拟交易模块

模拟交易模块，该模块作为交易执行层，负责维护 Agent 交易的核心账本。其主要设计点包括 Agent 的账户操作、仓位日志和生成 Agent 的资产快照。该模块将位于数据库层的“买入/卖出”封装为方法，在方法中保证 Agent 买卖事务的原子性和一致性，并暴露工具接口，将交易能力开放给 AI 决策层。

4.2.3 AI 决策模块

该模块是本系统核心的业务逻辑层，主要负责大语言模型的调度。其职责包括管理多 Agent 的实例、抓取数据组装 prompt、以及处理模型响应的结果。该模块通过行情数据模块获取 Agent 决策需要的信息，并通过暴露的 Tools 调用模拟交易模块的买卖接口，实现了数据、决策和交易的解耦。

如图 4.3 所示，以下是调用 AI 决策模块的核心流程。

用户配置并触发，用户首先在前端界面选择具体的 Agent 模型以及对应的风险偏好，随后点击“AI 操作”按钮激活流程。

前端转发请求，前端在感知到用户的点击事件后，立即将相关参数打包成请求，异步发送至后端。

后端组装提示词，后端接收到请求后，同步地获取市场实时数据并计算股票技术指标，随后将数据与对应的风险偏好提示词组合，构造 prompt 并调用大模型 API。

模型分析并决策，大模型接收提示词后，对当前市场数据与历史持仓进行推理分析，返回买入、卖出或观望的具体决策结果，附带执行方法的参数。

系统判断并交易，系统判断模型结果，若模型决定执行交易，则后端调用买卖工具执行交易，同时持久化记录交易流水与系统日志。

SSE 推送，最后无论是否交易，后端都会通过 SSE 管道将决策结果与日志实时推送到前端，更新日志列表和文本框，供用户分析 Agent 的决策理由。

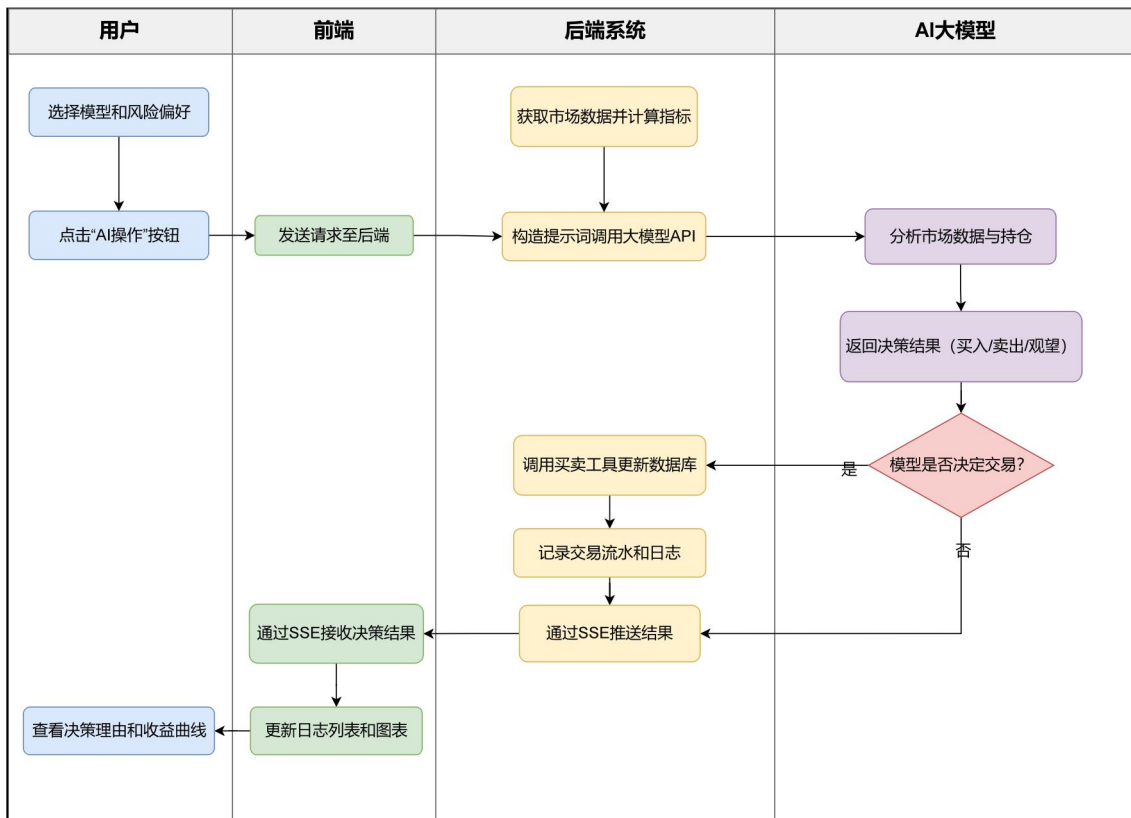


图 4.3 AI 决策模块流程图

4.2.3 斗兽场控制模块

该模块作为 Agent 在链上博弈的核心编排层，负责管理 Agent 对抗的生命周期。它按照设定的轮数和角色配置，控制每轮的交易顺序，协调 AI 决策模块和区块链交互模块完成代币交易，充当“中枢协调者”。

对斗兽场控制模块，笔者设计了如图 4.4 的核心流程：

前端配置，前端携带用户配置的参数请求后端“启动斗兽场”接口

配置落表，后端接收到请求后，将用户的配置记录插入配置表，设置初始状态为 RUNNING。

生成私钥、地址，调用 Web3j 的 Credentials.create 私钥生成方法，为三位 Agent 分配独立的以太坊地址与私钥，并将地址保存到数据库表的区块链地址字段。

初始化账户，操作系统托管地址向三位 AI 地址分别转入指定数量的 T1、T2、T3 代币，在区块链中转账通过 Web3j 调用 ERC20 合约 transfer 方法实现。

定义货币价格，在数据库表中插入初始价格记录。为了体现 AI 的交易表现，本系统中约定“SET”为“基准货币”，T1、T2、T3 相对 SET 的初始价格均为 1.0。

返回 response，前端跳转斗兽场监控页面展示 AI 实时收益；随后开始执行 AI 交易的循环，单次循环结束就将当前 Agent 的持仓和每个代币的价格返回给前端。

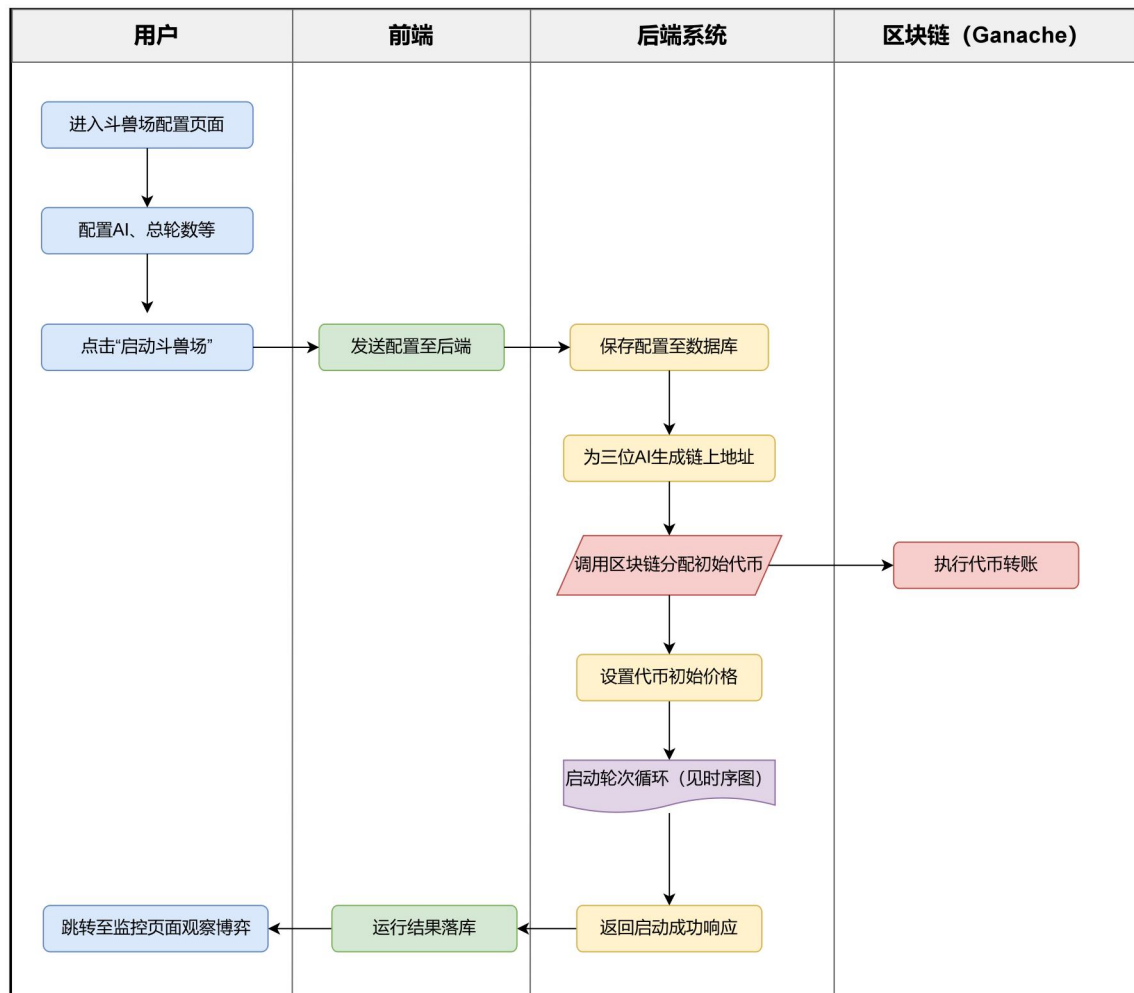


图 4.4 斗兽场模块流程图

4.2.4 区块链交互模块

该模块作为区块链网络 ABI 的适配层，负责隔离链上智能合约与本地业务系

统。包括部署代币合约、向指定地址转账初始代币、查询地址的代币余额、执行代币转账等操作，在最后一将交易日志和结果落入行情数据模块。

4.2.5 前端模块

该模块负责页面的展示和交互。该模块采用分屏与动态看板设计，其中股票交易页展示动态变化的行情表格、AI 选择控件、操作按钮、收益曲线和日志列表等；斗兽场页包含资产曲线、轮次信息、日志列表和交易记录表格。

4.2.6 模块间交互

各模块之间的交互关系可参考图 4.5，前端模块通过 HTTP 调用后端业务层的控制层；控制层根据请求调用行情数据模块、模拟交易模块、AI 决策模块或斗兽场控制模块；AI 决策模块在需要时调用模拟交易模块的工具；斗兽场控制模块在需要时调用 AI 决策模块和区块链交互模块；行情数据模块和区块链交互模块分别与外部 API 和 Ganache 节点通信。

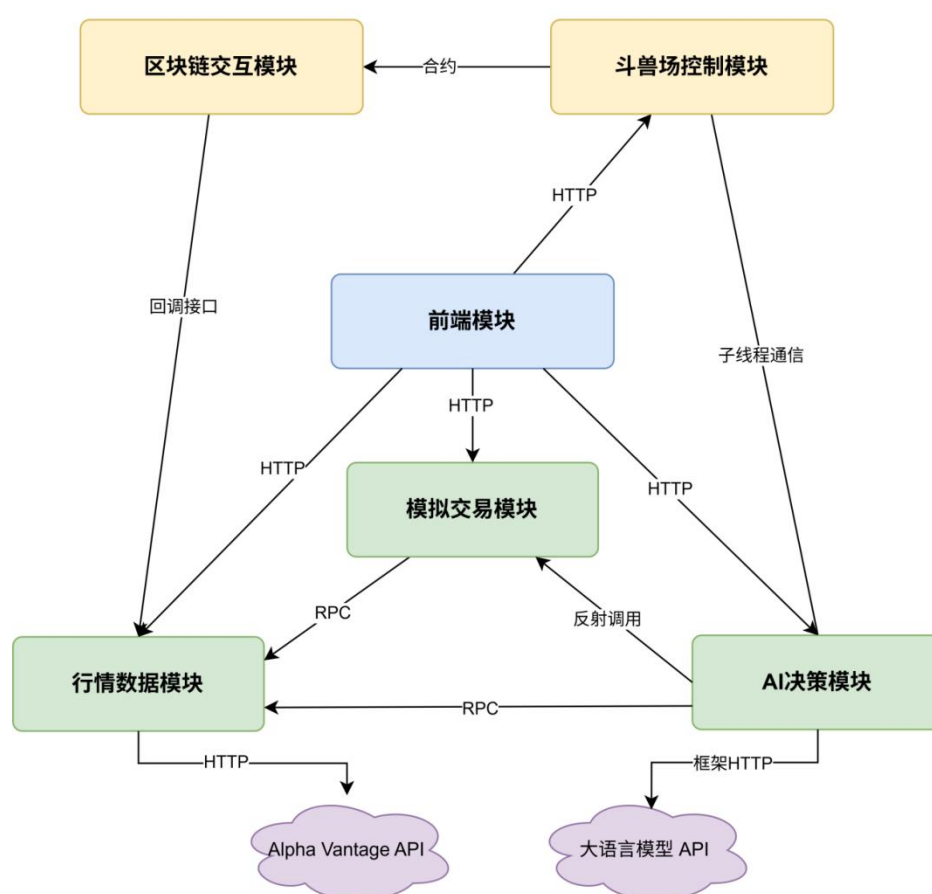


图 4.5 模块设计图

4.3 数据库表设计

本系统使用关系型数据库 MySQL 存储数据，以下列出主要的关系型数据表及

其字段，用 E-R 图表示数据表之间的关系。

4.3.1 E-R 图

图 4.6 是股票行情相关的 E-R 图，一个基本的股票(stock_info)可能关联着多条日线数据(stock_price)，它们之间通过 stock_id 连接

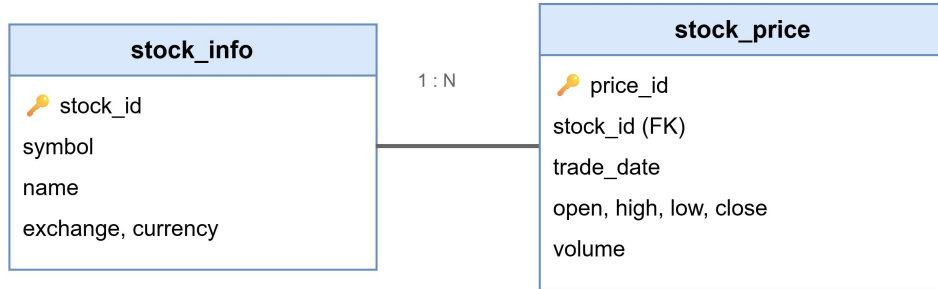


图 4.6 股票行情 E-R 图

图 4.7 是 AI 辅助交易的 E-R 图，其中单个 ai_trader 数据对应多个 ai_position 和 ai_decision_log 数据，表示一个交易员有多个持仓数据和决策日志，一个持仓数据也对应多个交易数据

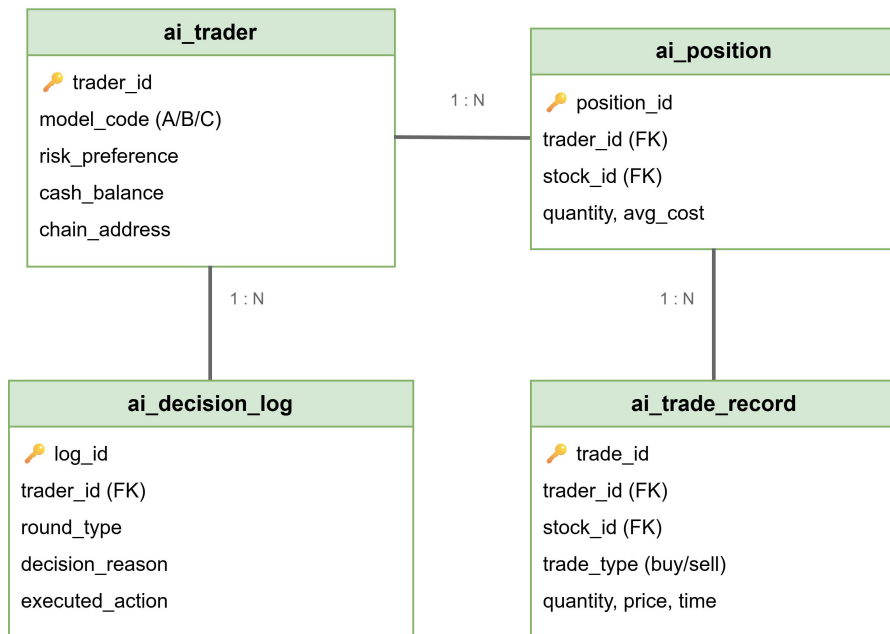


图 4.7 AI 辅助交易 E-R 图

图 4.8 是斗兽场的 E-R 图，其中单个回合数据对应多个代币价格和 Agent 交易数据，表示在单个回合中有产生了多次 Agent 交易和多种代币价格，而单个配置数据对应多轮回合数据则表明一次配置存在多轮回合。

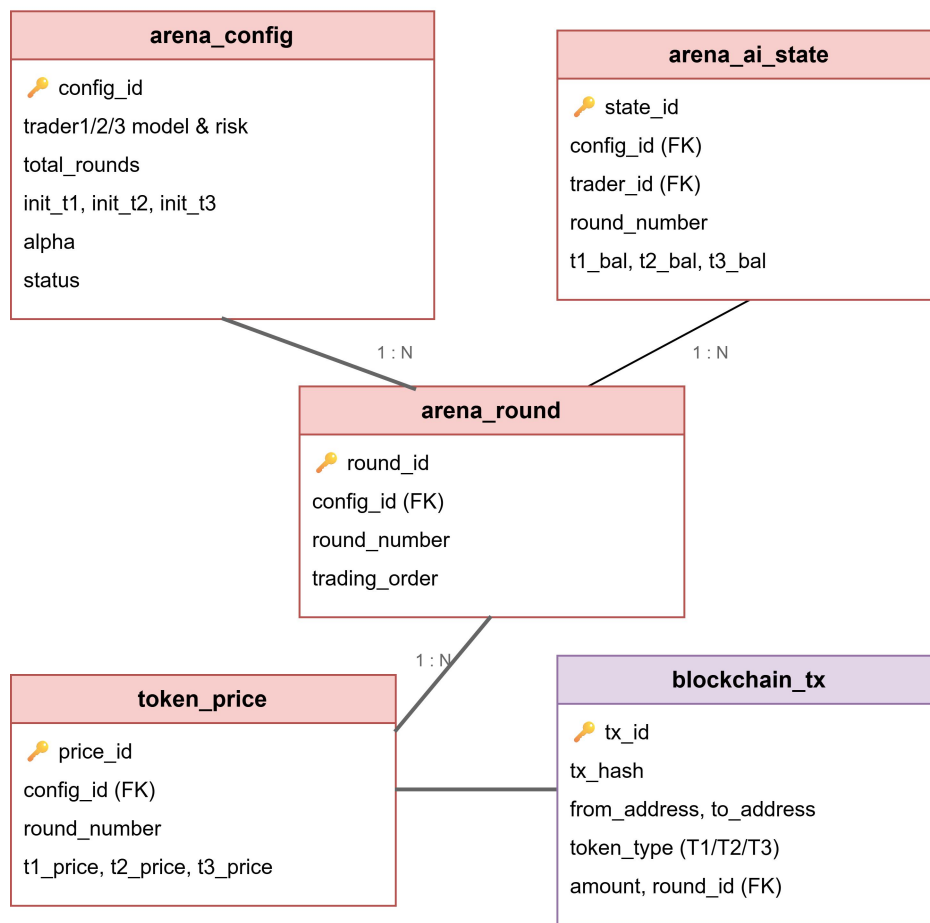


图 4.8 斗兽场相关 E-R 图

4.3.2 股票行情相关表

表 4.9 stock_info 表

字段名	类型	长度	含义
stock_id	INT	11	股票唯一自增 ID，主键
symbol	VARCHAR	20	股票代码（如 AAPL, 600000）
name	VARCHAR	100	股票中文或英文简称
exchange	VARCHAR	20	上市交易所（如 NASDAQ, SSE）
currency	VARCHAR	10	交易货币（如 USD, CNY）

表 4.10 stock_price 表

字段名	类型	长度	含义
price_id	BIGINT	20	价格记录唯一 ID，主键
stock_id	INT	11	股票 ID，外键关联 stock_info
trade_date	DATE	-	交易日期
open	DECIMAL	18,4	开盘价
high	DECIMAL	18,4	最高价
low	DECIMAL	18,4	最低价
close	DECIMAL	18,4	收盘价
volume	BIGINT	20	成交量

4.3.2 AI 辅助交易相关表

表 4.11 ai_trader

字段名	类型	长度	含义
trader_id	INT	11	交易员唯一 ID，主键
model_code	VARCHAR	50	大模型代码(如 ERNIE 5.0, GPT-4o-mini)
risk_preference	VARCHAR	20	风险偏好（如保守、激进、稳健）
cash_balance	DECIMAL	18,4	当前可用现金余额
created_at	TIMESTAMP	-	创建时间

表 4.12 ai_position

字段名	类型	长度	含义
position_id	BIGINT	20	持仓唯一 ID，主键
trader_id	INT	11	交易员 ID，外键关联 ai_trader
stock_id	INT	11	股票 ID，外键关联 stock_info

字段名	类型	长度	含义
quantity	INT	11	当前持仓数量
avg_cost	DECIMAL	18,4	持仓平均成本价

表 4.13 ai_trade_record

字段名	类型	长度	含义
trade_id	BIGINT	20	交易流水唯一 ID，主键
trader_id	INT	11	交易员 ID，外键关联 ai_trader
stock_id	INT	11	股票 ID，外键关联 stock_info
trade_type	VARCHAR	10	交易类型（buy：买入，sell：卖出）
quantity	INT	11	交易数量
price	DECIMAL	18,4	成交价格
trade_time	TIMESTAMP	-	交易成交时间

表 4.14 ai_decision_log

字段名	类型	长度	含义
log_id	BIGINT	20	日志唯一 ID，主键
trader_id	INT	11	交易员 ID，外键关联 ai_trader
round_type	VARCHAR	20	轮次类型（辅助交易 / 斗兽场轮次）
decision_reason	TEXT	-	AI 返回的自然语言决策理由
executed_action	VARCHAR	255	实际执行的操作指令
asset_value	DECIMAL	18,4	决策时的总资产评估值
created_at	TIMESTAMP	-	日志生成时间

4.3.3 斗兽场相关表

表 4.15 arena_config

字段名	类型	长度	含义
config_id	INT	11	配置唯一 ID，主键
trader1_model	VARCHAR	50	交易员 1 的大模型代码
trader1_risk	VARCHAR	20	交易员 1 的风险偏好设定
trader2_model	VARCHAR	50	交易员 2 的大模型代码
trader2_risk	VARCHAR	20	交易员 2 的风险偏好设定
trader3_model	VARCHAR	50	交易员 3 的大模型代码
trader3_risk	VARCHAR	20	交易员 3 的风险偏好设定
total_rounds	INT	11	斗兽场总运行轮数
init_t1	INT	11	代币 1 的初始分配数量
init_t2	INT	11	代币 2 的初始分配数量
init_t3	INT	11	代币 3 的初始分配数量
status	VARCHAR	20	运行状态（未开始、进行中、已结束）
created_at	TIMESTAMP	-	配置创建时间

表 4.16 arena_round

字段名	类型	长度	含义
round_id	BIGINT	20	轮次唯一 ID，主键
config_id	INT	11	配置 ID，外键关联 arena_config
round_number	INT	11	当前轮次序号
trading_order	VARCHAR	50	三位 AI 在本轮的交易顺序（如 1,3,2）
start_time	TIMESTAMP	-	本轮开始时间
end_time	TIMESTAMP	-	本轮结束时间

表 4.17 arena_ai_state

字段名	类型	长度	含义
state_id	BIGINT	20	快照唯一 ID，主键
config_id	INT	11	配置 ID，外键关联 arena_config
trader_id	INT	11	交易员 ID，外键关联 ai_trader
round_number	INT	11	当前结算轮次序号
t1_balance	INT	11	本轮结束后代币 1 的持仓量
t2_balance	INT	11	本轮结束后代币 2 的持仓量
t3_balance	INT	11	本轮结束后代币 3 的持仓量
set_value	DECIMAL	18,4	折算后的 SET 代币总资产值

表 4.18 token_price

字段名	类型	长度	含义
price_id	BIGINT	20	价格记录唯一 ID，主键
config_id	INT	11	配置 ID，外键关联 arena_config
round_number	INT	11	当前轮次序号
t1_price	DECIMAL	18,4	代币 1 相对于 SET 的动态价格
t2_price	DECIMAL	18,4	代币 2 相对于 SET 的动态价格
t3_price	DECIMAL	18,4	代币 3 相对于 SET 的动态价格

表 4.19 blockchain_tx

字段名	类型	长度	含义
tx_id	BIGINT	20	流水唯一自增 ID，主键
tx_hash	VARCHAR	66	测试链上的交易哈希值
from_address	VARCHAR	42	转出方钱包地址
to_address	VARCHAR	42	转入方钱包地址

字段名	类型	长度	含义
token_type	VARCHAR	10	代币类型（T1, T2, T3, SET）
amount	DECIMAL	24,4	转账代币数量
round_id	BIGINT	20	关联的斗兽场轮次 ID，外键关联
created_at	TIMESTAMP	-	区块链交易打包时间

4.4 本章小结

本章阐述了本系统的总体设计，笔者在架构上采用前后端分离、离线在线数据分离的四层结构。前端 Vue 3 展示页面，后端 SpringBoot 负责处理业务的逻辑，MySQL 存储数据，Ganache 测试链保存代币资产记录。在功能上，本系统划分为行情数据、模拟交易、AI 决策、斗兽场控制、区块链交互和前端界面六个模块。数据库中，笔者设计了四类表，分别支持了股票行情、AI 交易、斗兽场博弈和链上交易记录的存储。

5 模拟交易所系统实现

本章将在总体设计的基础上，对各功能模块实现上的细节进行具体的阐述，包括行情数据的查询与缓存策略、AI 决策模块的提示词拼接加载与模型调用、模拟交易的工具方法实现、斗兽场的竞争排序与代币兑换流程，以及前端与后端的交互方式。对于关键的逻辑，笔者将给出代码片段或伪代码。

5.1 行情数据模块实现

5.1.1 股票数据查询与缓存

本系统固定配置了几只美股作为交易对象，包括 IBM、AAPL、TSLA、MSFT 等。股票代码清单硬编码于 SpringBoot 配置文件 application.yml 中。

```
trading:
  allowed-stocks:IBM,AAPL,TSLA,MSFT
```

后端在启动时读取该配置，然后初始化 stock_info 表，若表中无记录则新建。之所以采用配置的方式导入交易对象，是为了后续将本地配置升级到远程配置服务，如：Nacos、Apollo 等，以实现在项目上线后动态调整收录股票的功能，在该阶段则暂时配置在本地。

5.1.2 股票技术指标的计算

笔者并没有调用 Alpha Vantage 的指标 API，而是在后端进行了技术指标的计算，主要用到了 org.apache.commons.math3 库中的统计工具类^[22]。

移动平均线：获取最近 N 个交易日的收盘价序列 C_i ，调用 StatUtils.mean(double[])方法计算算术平均数，公式（5.1）如下

$$MB = \frac{\sum_{i=1}^N C_i}{N} \quad (5.1)$$

相对强弱指标：按标准公式计算，其中 RS 为 N 日内上涨幅度均值 AvgGain 与下跌幅度均值 AvgLoss 的比值，公式（5.2）如下。

$$RSI = 100 - \frac{100}{1 + RS}, \quad \text{其中} \quad RS = \frac{AvgGain}{AvgLoss} \quad (5.2)$$

布林带：中轨 MB 为 N 日收盘价的移动平均线 MB，带宽是标准差通过 DescriptiveStatistics 类求得，上轨（Upper Band）与下轨（Lower Band）则分别在中轨的基础上加减两倍标准差，公式（5.3）中的 K 通常取值为 2。

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (C_i - MB)^2}{N}}$$

$$\begin{aligned} UpperBand &= MB + K \cdot \sigma \\ LowerBand &= MB - K \cdot \sigma \end{aligned} \quad (5.3)$$

指标计算类设计为无状态工具类，方法签名示例：

```
public class IndicatorCalculator {
    public static double sma(List<Double> prices, int period) { ... }
    public static double rsi(List<Double> prices, int period) { ... }
    public static BollingerBand bollinger(List<Double> prices, int period) { ... }
}
```

以上各种方法都在 AI 决策时调用，调用结果用于辅助 AI 进行推理；本系统并不将指标结果持久化，以维护数据结构的简洁和实效性

5.2 AI 决策模块实现

5.2.1 模型配置与加载

本系统共集成了三个不同厂商的大语言模型，代号 A、B、C 分别对应百度 ERNIE 5.0、阿里通义千问 Max、OpenAI GPT-4 mini。各模型的 baseUrl、apiKey 及模型名称参数配置于 application.yml 中：

```
langchain4j:
  models:
    ernie:
      base-url: https://qianfan.baidubce.com/v2
      api-key: ${ERNIE_API_KEY}
      model-name: ernie-5.0
    qwen:
      base-url: https://dashscope.aliyuncs.com/compatible-mode/v1
      api-key: ${QWEN_API_KEY}
      model-name: qwen-max
    gpt4:
      base-url: https://api.openai.com/v1
      api-key: ${OPENAI_API_KEY}
      model-name: gpt-4o-mini
```

api-key 通过本地的环境变量注入，这样配置文件提交至 Git 仓库后就不会暴露敏感信息造成 token 盗用。SpringBoot 在启动时读取上述配置连接模型厂商 api，为每个模型接口构建 TradeModel 实例，再通过 @bean 注解注入 Spring 容器。AgentFactory 创建 AIServices 代理，该工厂类根据前端传入的模型代号返回相应的代理对象，代理再通过反射调用 TradeMode 中的交易接口，这样就实现了用一个对象（代理对象）管理多个模型实例的收束逻辑。

5.2.2 提示词模板管理

系统提示词存储于 main/resources/prompts/目录下的单个文本文件中。文件内容为纯文本，对模型起到总领性提示的作用，在每次交易中提示模型将结果结构

化输出。`prompt` 具备通用性，通过 `String.format` 插入余额、持仓情况和风险偏好等：

你是一个专业的股票投资决策专家。请根据以下信息做出投资决策。

【你的账户信息】

- 模型编号: `{modelId}`
- 模型名称: `{modelName}`
- 当前余额: `${deposit}`
- 投资风格: `{riskPreferenceName}`
- 风险偏好描述: `{riskPreferenceDesc}`

【当前持仓情况】

`{positionData}`

【投资指导】

`{riskGuidance}`

【可选股票及其特征数据】

`{stockFeatures}`

【可用工具】

- `buyStock(modelId, modelName, stockName, stockCode, amount)`: 买入股票
- `sellStock(modelId, modelName, stockName, stockCode, amount)`: 卖出股票

【输出格式要求】

请严格按照以下 JSON 格式输出你的投资决策，不要添加任何其他说明文字：

```
{
  "summary": "决策总结，简要说明整体策略",
  "tradeDecisions": [
    {
      "action": "BUY/SELL/HOLD",
      "stockCode": "股票代码",
      "stockName": "股票名称",
      "amount": 交易数量,
      "reason": "交易原因"
    }
  ],
  "marketAnalysis": {
    "overallTrend": "整体市场趋势判断",
    "stockAnalyses": [
      {
        "stockCode": "股票代码",
        "stockName": "股票名称",
        "technicalAnalysis": "技术分析",
        "trend": "趋势判断",
        "supportLevel": "支撑位",
        "resistanceLevel": "阻力位"
      }
    ]
  }
}
```

```
},  
"riskAssessment": {  
  "riskLevel": "LOW/MEDIUM/HIGH",  
  "riskDescription": "风险说明",  
  "riskControlMeasures": "风控措施",  
  "expectedReturn": "预期收益",  
  "maxAcceptableLoss": "最大可承受损失"  
}  
}
```

【决策要求】

1. 分析每支股票的技术指标和趋势
2. 根据你的{riskPreferenceName}风格，决定是否买入、卖出或持有
3. 考虑当前持仓情况，避免过度集中或重复持仓
4. action 只能是 BUY、SELL 或 HOLD 之一
5. 如果决定交易（BUY/SELL），请同时调用相应的工具执行交易
6. 请详细填写每个字段，确保 JSON 格式正确
7. **重要提示**：买入股票时，请确保购买数量所需的总金额不超过当前余额\${deposit}，否则交易将会失败。计算方式为：购买数量 × 当前股价 ≤ 余额，如果金额超过了当前余额导致买卖失败，可以调小买入金额再次进行买卖

每次构建 AiServices 代理时，AgentFactory 会读取对应的文件内容并将其设置为 SystemMessage。外置的提示词具备可热修改的特性，便于笔者在实验过程中快速调整 AI 的引导方向，而无需重新编译代码。

5.2.3 单次决策的无状态设计

在 AI 辅助交易模块中，每次决策都为独立的无状态的 Session。前端点击“AI 决策”按钮后，后端将构造全新的 TradingContext 对象，其中包含当前的市场快照和 AI 持仓数据，拼接到 prompt 中提交给模型进行推理。推理结束，展示到前端后推理过程则被抛弃。此设计避免了复杂的上下文管理，也很大程度上降低了对话带来的高昂 token 费用。

斗兽场模块的设计则不一样。Langchain4j 提供了管理上下文记忆的接口，笔者使用了 MessageWindowChatMemory 滑动窗口容器存储会话记忆。每个 AI 在博弈过程中维护这样的短期记忆容器，执行先进先出的淘汰策略，仅记录离自己最近的五轮操作摘要（如“上轮用 100 T1 换得 52 T2，原因是 RSI 显示超卖”），该操作摘要是由买卖接口的日志直接注入所有 Agent 的记忆容器中。这样，AI 就能够在决策时参考自身和其他 Agent 的历史行为，增加推理的连贯性。短期记忆也不会跨轮次进行无限累计，避免了提示词的膨胀。

5.2.4 线程池配置

在本系统的多线程场景中，调用大模型 API 响应时间从数秒到数十秒不等，属于典型的 I/O 密集型任务。该类型任务可以通过增加并发度或调用异步非阻塞

rpc 进行优化，而本系统的大模型接口并不支持异步调用，只能同步阻塞，等待其返回数据。所以为了充分利用系统资源，笔者结合了机器性能（12 核）和任务特性（极高的调用耗时），为 AI 决策模块配置了独立的线程池，并设计了定制化的参数：

```
@Bean("aiExecutor")
public ExecutorService aiExecutor() {
    return new ThreadPoolExecutor(
        12, // 核心线程数, cpu 核心数量为12
        60, // 最大线程数
        60L, TimeUnit.SECONDS, // 空闲线程存活时间
        new LinkedBlockingQueue<>(), // 任务队列容量
        new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略
    );
}
```

部署本系统的机器 cpu 内核数为 12，考虑到大模型 API 的耗时极长，且并发限制较为宽松，笔者配置了核心线程数为 12，最大线程数为 60 和链表型的任务队列。其中核心线程对应 cpu 内核，理想情况下一个内核执行一个任务，60 的最大线程数提升系统的并发能力，如果出现 60 个线程都被占用的情况，多余的任务会暂存到无限长的任务队列中，有效缓冲瞬时的流量高峰；拒绝策略为 CallerRunsPolicy，防止任务丢失。经压力测试，该配置足以支撑 5qps 的并发量，足以满足系统对 AI 决策的并发需求。

5.3 模拟交易模块实现

5.3.1 买卖工具方法

工具方法本质上就是由 Java 实现的代码逻辑，本系统的 buy/sell 方法都加上了 @Tool 注解，由 AiService 调用反射执行。下面给出买入方法，完整实现如下：

```
@Tool("买入指定股票。symbol 为股票代码，amount 为买入股数。")
@Transactional(value = "Exception")
public String buy(String symbol, double amount) {
    StockInfo stock = stockInfoRepository.findBySymbol(symbol)
        .orElseThrow(() -> new IllegalArgumentException("无效股票代码"));
    double currentPrice = stockPriceRepository.findTopByStockIdOrderByTradeDateDesc(stock.getId())
        .orElseThrow(() -> new IllegalStateException("无行情数据"));
    double requiredCash = currentPrice * amount;
    AiTrader trader = getCurrentTrader();
    if (trader.getCashBalance() < requiredCash) {
        return "买入失败：现金余额不足";
    }
}
```

```

trader.setCashBalance(trader.getCashBalance() - requiredCash);
aiTraderRepository.save(trader);
AiPosition position = aiPositionRepository.findByTraderIdAndStockId(
trader.getId(), stock.getId())
    .orElse(new AiPosition(trader, stock, 0, 0.0));
double newQuantity = position.getQuantity() + amount;
double newAvgCost = (position.getAvgCost() * position.getQuantity() +
requiredCash) / newQuantity;
position.setQuantity(newQuantity);
position.setAvgCost(newAvgCost);
aiPositionRepository.save(position);
AiTradeRecord record = new AiTradeRecord(trader, stock, TradeType.BUY,
amount, currentPrice);
aiTradeRecordRepository.save(record);
return String.format("已买入 %.0f 股 %s, 成交价 %.2f, 消耗现金 %.2f",
    amount, symbol, currentPrice, requiredCash);
}

```

5.3.2 AI 交易员初始化

在本项目初始化时，笔者为每位 AI 交易员分配了 10,000 美元的初始现金。初始化逻辑位于 `AiTraderService.createTrader`，该方法被 `@PostConstruct` 修饰，在依赖注入完成后，Bean 正式投入使用前执行。另一部分，前端在启动斗兽场模块时，将引导用户对 AI 的模型和风险偏好进行自由组合，并且设置 AI 的初始现代币数量（默认为 1,000 单位），这在数据层对应着 `ai_trader` 表中的记录。

5.4 区块链斗兽场模块实现

5.4.1 角色配置与启动

在本系统中，用户可以对斗兽场模拟进行配置，比如可以选择 Agent 的底层模型和提示词风险偏好（稳妥/适中/激进），也可以设置基本的参数，包括 AI 的数量、总的运行轮数和 Agent 的初始 Token 数量，默认是 T1、T2、T3 各分配 1,000 单位，托管地址预留 10,000 单位，模拟开始后，所写的配置将会加载到线程中。

5.4.2 竞争排序与轮转执行

本系统中，笔者构造了 `ArenaRoundExecutor` 类控制每轮博弈的流程，分为两个决策阶段。核心逻辑如下：

```

public void executeRound(Long configId, int roundNumber) {
    ArenaConfig config = configRepository.findById(configId).get();
    List<AiTrader> traders = getTradersByConfig(config);
    List<AiTrader> racingOrder = Collections.synchronizedList();
    CountdownLatch racingLatch = new CountdownLatch(traders.size());
    for (AiTrader trader : traders) {
        arenaExecutor.submit(() -> {

```

```

        try {
            buildInitialIntent(trader, config);
            racingOrder.add(trader);
        } finally {
            racingLatch.countDown();
        }
    });
}
try {
    racingLatch.await(10, TimeUnit.SECONDS);
    for (AiTrader trader : racingOrder) {
        TradingContext flashCtx = buildContextForArena(trader, config
    );

        TradeAction action = trader.confirmDecision(flashCtx);
        if (action.isProceed()) {
            executeTraderTurn(trader, config, roundNumber, flashCtx);
        } else {
            log.info("第 {} 轮: 交易员 {} 观察到价格变动, 决定放弃交易", roundNumber, trader.getId());
        }
    }
    saveRoundSnapshot(config, roundNumber, racingOrder);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}

```

在任务的一开始，程序会根据用户的配置构建对应的 `AiTrader`；

第二步是创建线程安全队列，该列表的 `add` 操作会被加上 `synchronized` 关键字修饰，同步锁机制保证了其多线程环境下的有序插入，该列表的顺序表示 `trader` 执行交易的顺序；

第三步，所有 `AI` 会同时启动第一轮预决策，这轮决策的速度决定了他们在本轮交易的顺序，这也符合真实场景下，反应更敏捷的交易员能够更快做出判断、执行交易；

第四则是 `AI` 参考预决策与现实情况（同轮次前面的交易会影响 `Token` 的价格），执行二次决策，在差异不大时调用 `buy/sell` 接口执行交易任务；

最后，保存交易日志，并将日志写入其他 `Agent` 的 `ChatMemory` 中，使其具备上下文感知能力，重启循环，直到达到用户配置的轮数结束。

图 5.1 是上述三个线程与区块链模块、大语言模型和主线程交互的时序图

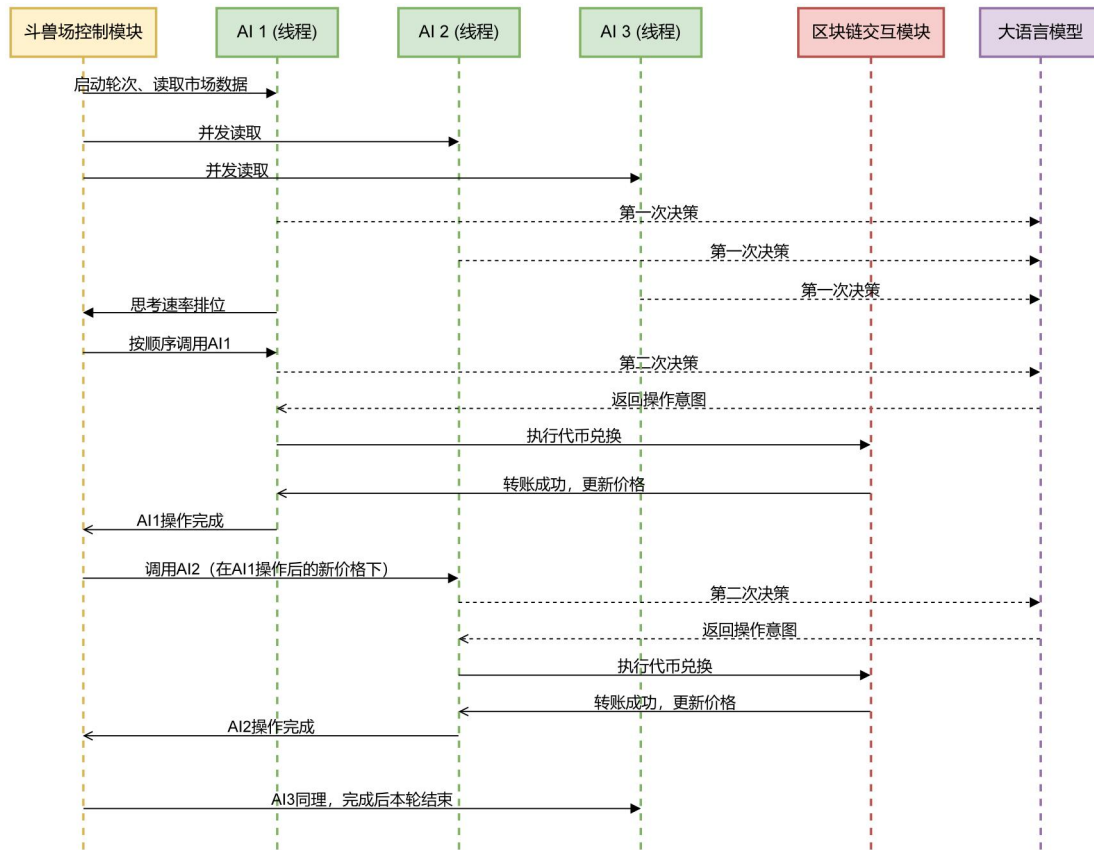


图 5.1 斗兽场交易时序图

5.4.3 代币兑换与价格更新

AI 执行最终决策的方法是 `executeTraderTurn`，具体有以下几步：

调用 AI 决策模块，传入预决策结果和当前市场数据（各代币最新价格、AI 持仓），模型自动调用 `buy/sell` 工具接口。在本场景 `buy/sell` 的工具中，若模型选择用 T1 兑换 T2，则传入 `fromToken`、`toToken`、`amountIn` 等参数，根据当前价格公式（5.4）计算出能够兑换的 `toToken` 的量 `amountOut`。

$$amountOut = amountIn \times \frac{priceOf(fromToken)}{priceOf(toToken)} \quad (5.4)$$

在链下确定了兑换货币的数量后，则开始执行链上转账：调用 `fromToken` 合约的 `transfer` 方法，从 AI 地址划走 `amountIn` 至系统托管地址；再调用 `toToken` 合约的 `transfer`，从托管地址划转 `amountOut` 至 AI 地址；在链下，系统将记录交易日志和余额快照，并把所有 DB 读写操作包裹在同一数据库事务中，以保证转账操作的原子性。

转账成功后，`fromToken` 和 `toToken` 的相对数量会发生改变，同时这对 Token 的价格也会发生改变，本系统选择使用线性价格冲击公式计算两种 Token 的价格

波动，如公式（5.5）。

$$P_{new} = P_{old} \cdot \left(1 + \alpha \cdot \frac{amountIn}{S} \right) \quad (5.5)$$

其中 S 为该代币总供应量， α 取配置文件中的 `arena.price.alpha` 值（默认 0.5）。`amountIn` 为本次兑换付出的代币数量， P 则是价格。卖出操作则可以视作净买入量为负值，公式自然同样适用。

更新 `token_price_history` 表，记录 `fromtoToken` 的新价格；保存 AI 的决策日志至 `ai_decision_log` 表，包括操作内容与决策理由；并记录链上交易哈希至 `blockchain_tx` 表。

每轮交易后价格都会变动，这将影响之后同轮 AI 的判断，笔者在上文设计的二次决策解决了该问题。

5.4.4 链上交互与私钥管理

本系统的链上交互主要由 Web3j 负责，手动调用或者在编译时启动 `web3j-maven-plugin` 插件，该插件会自动根据 Solidity 源码生成 Web3j 的合约封装类。下面展示部分转账操作的代码片段：

```
TradingToken tokenContract = TradingToken.load(contractAddress, web3j, credentials, new DefaultGasProvider());
Receipt receipt = tokenContract.transfer(toAddress, amount).send();
```

`DefaultGasProvider` 是 Web3j 提供的默认策略类，主要起到计算 `gasFee` 的作用，其在 Ganache 测试环境下默认将 `gasFee` 设为零，且 `gasLimit` 设为合约执行所需的最小值。为了简化系统复杂性，本系统暂不研究 `gasFee` 对 AI 决策的影响，所以直接使用该默认类，并没有额外配置。

笔者在系统初始化时，创建了一个汇款“操作员”——系统托管方，该地址的私钥存放于本地环境变量中，该私钥对应的地址是部署代币合约时的初始持有者，拥有全部数量代币，负责中转 AI 交易员的 Token 交易。

5.5 前端与后端的交互实现

5.5.1 实时数据推送

本系统涉及许多实时数据推送的场景，比如 AI 决策日志和资产曲线的更新。笔者在技术选型时，调研了工程上常用的 Server 推送协议：WebSocket 和 SSE，考虑到 WebSocket 维持全双工通道会消耗更多的服务器资源，而且单向推送也完全满足本系统需求，所以笔者选用了更加轻量成本更加低廉的 SSE。前端使用 `EventSource` 对象连接后端的 `/api/stream` 接口组。后端在每一轮决策完成时，都会向 SSE 通道写入最新快照数据，前端通过监听到的 `message` 事件，增量更新 `ECharts`

图表和对话框，展示 AI 的资产曲线和决策思路。

例如，在斗兽场运行期间，前后端会维护一条 SSE 长连接接收 AI 资产的更新；股票交易页的 AI 单次决策结果同样通过 SSE 的 HTTP 连接返回，避免前端轮询造成不必要的资源浪费。

5.5.2 页面组件结构

前端两个标签页是独立的 Vue 单文件组件，分别为 StockTrading.vue 和 Arena.vue。笔者考虑到本地计算资源的有限，所以切换标签页时，Vue Router 会自动销毁离开页面的组件实例，也就是自动停止该页面的 SSE 连接和未完成的 HTTP 请求。这两个页面各自维护数据状态，具备隔离性。

5.5.3 配置页面的实现

斗兽场的配置页面也是一个独立的 Vue 组件，包含三个 AI 角色的下拉选择器、轮数输入框、初始代币分配输入框。提交时将配置数据通过 POST 接口推送至后端，后端创建 arena_config 记录并返回配置 ID。随后页面自动跳转到斗兽场监控页 Arena.vue，通过配置 ID 订阅后端返回的 SSE 流，在该步骤之后，后端就通过该流传输数据，前端使用 Echart 进行展示。

5.6 异常处理与日志

本系统使用 SpringBoot 提供的@ControllerAdvice 等一系列注解，该类注解就像是一张拦截网，捕获 Controller 中的数据进行处理。笔者使用 @ExceptionHandler(Exception.class)注解，捕获并处理 IllegalArgumentException、API 调用超时等异常，以统一的 JSON 返回错误信息给前端。

关键业务操作均通过 SLF4J 记录日志，默认为 info 级别；AI 决策时，提示词内容与模型调用工具返回以 DEBUG 级输出，便于调试。同样是考虑到了资源的有限，日志文件每日滚动，保留周期为三天。

6 模拟交易所系统测试与分析

本章在前文设计与实现的基础上，对系统进行功能验证与运行效果分析。测试分为两个场景：股票辅助交易的 AI 决策效果，以及区块链斗兽场的多 Agent 博弈过程。通过实际运行截图与结果数据，展示系统各模块的协同工作情况。

6.1 测试环境

系统部署于本地开发环境，具体配置如表 6.1 所示。

表 6.1 开发环境表

项目	配置
CPU	Intel Core i5-12400H（12 核 16 线程）
内存	16 GB
JDK 版本	17
Node.js 版本	18.16.0
数据库	MySQL 8.0
区块链环境	Ganache 2.7.1
大模型 API	ERNIE 5.0 / qwen-max / GPT-4o-mini

6.2 股票辅助交易功能测试

6.2.1 行情数据展示

系统启动后，前端股票交易页面展示若干只美股的基本信息与 K 线图，如图 6.2 所示。行情数据从 Alpha Vantage 拉取后缓存。



图 6.2 行情数据展示图

6.2.2 AI 单次决策过程

用户在界面上选择 AI 模型（以通义千问 qwen-max 为例）和风险偏好（适中），点击“AI 决策”按钮触发单次辅助交易。系统将市场快照与 AI 持仓数据打包为提示词发送给大模型，模型推理后返回决策理由与操作指令。

表 6-3 记录了某次决策中，同一市场条件下三种风险偏好 AI 的不同行为。

表 6.3 不同风险偏好决策表

AI 风险偏好	决策理由摘要	操作	资产变化
保守	大盘方向不明，IBM 技术指标中性，选择持有现金观望	无操作	0
适中	AAPL 收盘站稳 20 日均线上方，RSI 处于中位，适度建仓	买入 AAPL 10 股	-2,216 USD
激进	TSLA 连涨三日动能强劲，MACD 金叉确认，加仓博取短线收益	买入 TSLA 20 股	-5014 USD

由表 6-2 可见，保守型 AI 在当前市场条件下选择观望，而激进型 AI 倾向于捕捉短期动量进行交易。三种 AI 的行为差异体现了提示词模板对模型决策方向的有效引导。



图 6.4 保守型 Agent 单次决策结果

保守型 AI 的决策报告中“当前市场环境下建议保持谨慎”，它提出了总仓位控制在 30%以内、优先保护本金安全、5%收益后及时止盈等风险控制原则。

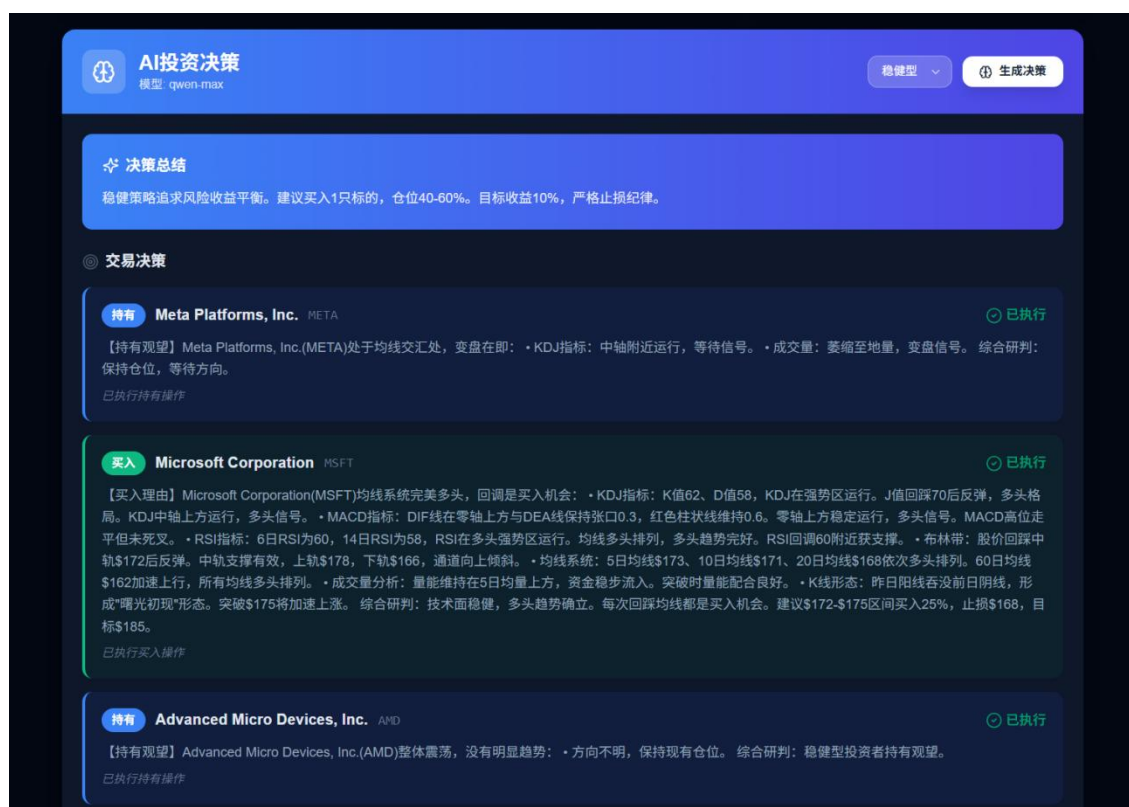


图 6.5 稳健型 AI 单次决策结果

在具体操作上，模型通过 KDJ、MACD、RSI 等技术指标做出判断，选择了执行卖出 APPL，买入 GOOGL 和持有 META。

稳健型 AI 的决策声明了谨慎的态度，计划买入 1 支持仓并表明只追求 10% 的收益率。同样是根据各种技术指标进行判断，最终决定买入 MSFT

激进型 AI 对短期的价格动能表现出了高度的敏感，具体表现为计划重仓买入 3 支股票，并维持 60-80% 的仓位，目标是追求 20% 以上收益，如图 6.6 所示。

它识别到 TSLA 的 MACD 指标出现金叉，并且连续三日放量上涨，倾向于博取短线反弹收益。模型随即调用买入工具进行下单。其决策理由中使用了“右侧上车”“动能确认”等偏激进的表述，与前两种偏好形成鲜明对比。

从三张截图的对比可以直观看出，面对完全相同的市场数据，三种风险偏好的 AI 给出了不同的交易行为和决策理由：保守者按兵不动，适中者小幅试探，激进者大举进攻。这一点与前文表 6.3 的结论一致，进一步体现了提示词模板对模型决策风格的有效引导。

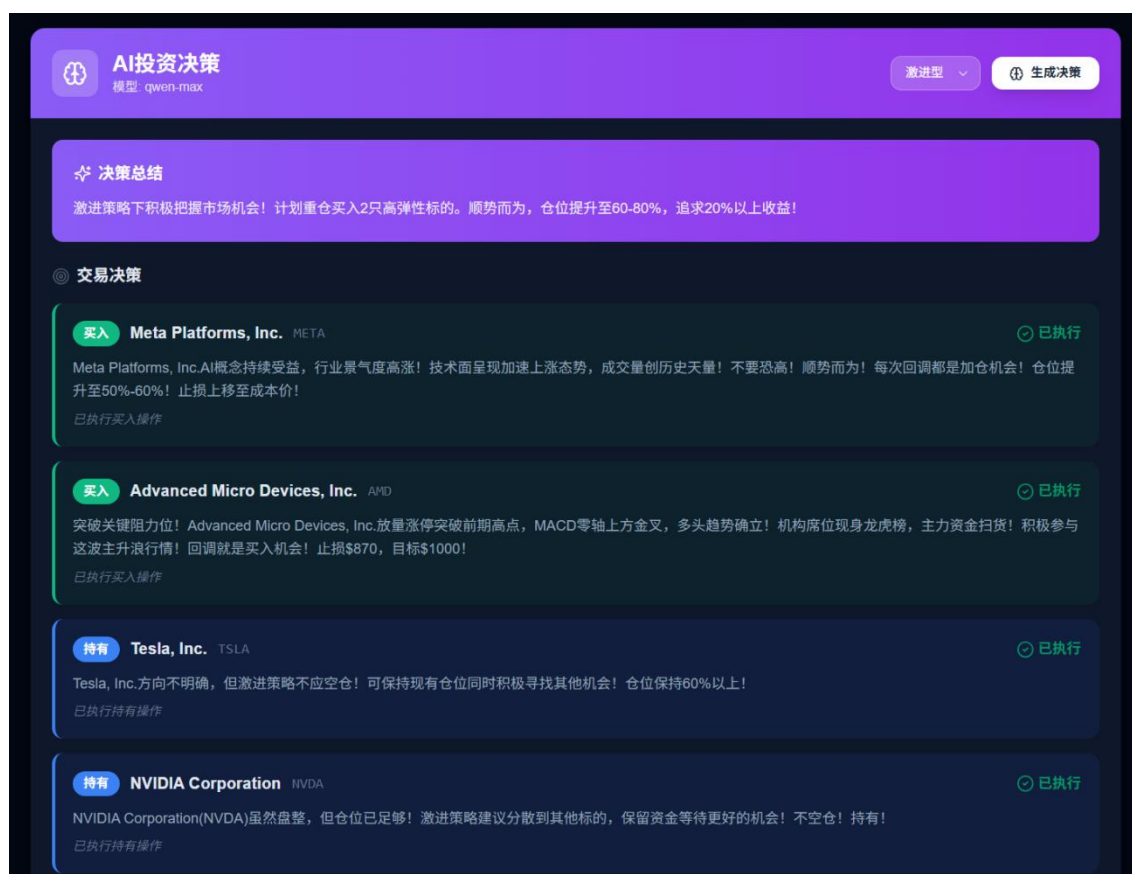


图 6.6 激进型 AI 单次决策结果

6.2.3 资产曲线展示

每次决策结束后，系统根据 AI 现金余额与持仓股票市价计算收益总值，使用者可观察其在多次决策后的收益差异。图 6-7 展示 AI 的总收益以及其收益组成。



图 6.7 AI 收益图

为了进一步分析 AI 执行的决策，本系统在股票维度下钻持仓，图 6-7 展示 AI 持仓的明细，通过卡片形式展示 AI 在各支股票的占比。



图 6.7 持仓明细图

6.2.4 股市行情分析功能验证

AI 在做出交易决策的时候会先进行股市行情的判断，这一判断可能会对用户起到一定的参考价值，所以笔者选择将其展示至决策信息后。

系统将事先计算好的技术指标数据（包括 5 日、10 日、20 日均线，相对强弱指标 RSI，以及布林带上下轨位置等）同近期收盘价数据，一同注入大语言模型的提示词中。系统要求模型不论是否执行交易，都要对每只股票进行解读，并输出结构化的分析报告。

表 6-8 截取了三位不同风险偏好的 AI 对同一只股票“AAPL”的技术分析结论。可以见到，他们通过布林带、均线等技术指标，对 AAPL 做出了各自的判断，用户可以根据这些分析，判断自己的决策考虑是否周到。

图 6.8 不同风险偏好 AI 对标的 AAPL 的行情分析

AI 角色	技术分析内容摘录
保守型	AAPL 当前价格处于 20 日均线下方，布林带正在收窄，方向尚不明朗。成交量并未放大，倾向于认为此处整理尚未结束，存在破位下行风险。
适中型	股价在触及布林带下轨后获得支撑，出现了初步的反弹迹象。但 20 日均线依然构成上方压力，需观察能否有效突破。短期看震荡筑底。
激进型	RSI 指标已从超卖区回升，且 5 日均线有金叉 10 日均线的迹象。对此

AI 角色

技术分析内容摘录

我解读为右侧买点正在形成，可以轻仓位博取反弹收益。

由表 6-8 可见，即便面对完全相同的行情数据，不同偏好的 AI 也给出了不尽相同的解读。保守型更关注风险信号，适度型寻求平衡确认，而激进型则对反转信号非常敏感。这说明了风险偏好在 AI 的行情分析风格上也起到了有效的作用，前端展示 AI 市场分析如图 6-9 所示。



图 6.9 市场分析图

6.2.5 交易风险评估功能验证

在分析行情之后，AI 会进一步结合其当前的虚拟持仓和现金状况，对潜在的交易进行风险评估。这一环节是连接“市场分析”与“交易执行”的决策枢纽，也能够让用户拥有不同视角的风险视野，笔者在提示词中添加风险评估的结构化输出。

```
"riskAssessment": {
  "riskLevel": "LOW/MEDIUM/HIGH",
  "riskDescription": "风险说明",
  "riskControlMeasures": "风控措施",
  "expectedReturn": "预期收益",
  "maxAcceptableLoss": "最大可承受损失"
}
```

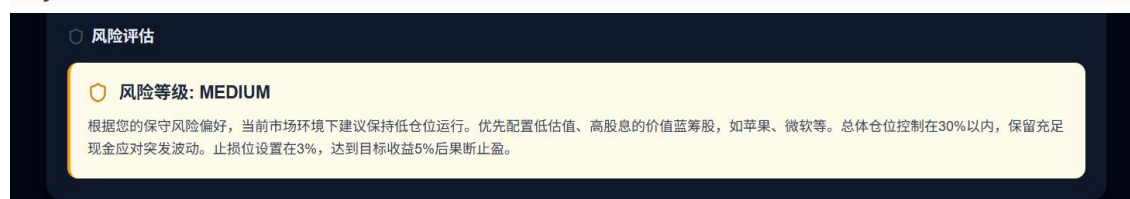


图 6.10 AI 风险评估日志展示

AI 整合技术状态、自身持仓成本和整体投资组合结构后，最终在决策日志中输出了图 6.10 的风险判断。

这一输出结果证明了 AI 能够综合技术指标进行复合型风险评估，而非仅仅依赖 K 线对单只股票进行买卖。其输出的风险等级建议，具备基本的合理逻辑，对使用者有一定的参考意义。

6.3 区块链斗兽场功能测试

6.3.1 斗兽场配置与启动

使用者在斗兽场配置页面分别为三位 AI 选择模型与风险偏好。配置页面如图 6-11 所示，用户可以在这里新增、删除 AI 交易员数量，配置每个 AI 的模型和风险偏好；’也可以配置三种测试代币的基础数量和总的回合轮数。

表 6.12 斗兽场测试配置

• AI 角色	模型	风险偏好	初始 T1	初始 T2	初始 T3
AI-A	通义千问 Max	保守	1,000	1,000	1,000
AI-B	ERNIE 5.0	适中	1,000	1,000	1,000
AI-C	GPT-4o-mini	激进	1,000	1,000	1,000

本次测试选用的测试模型、风险偏好和初始 Token 如表格 6.12 所示。

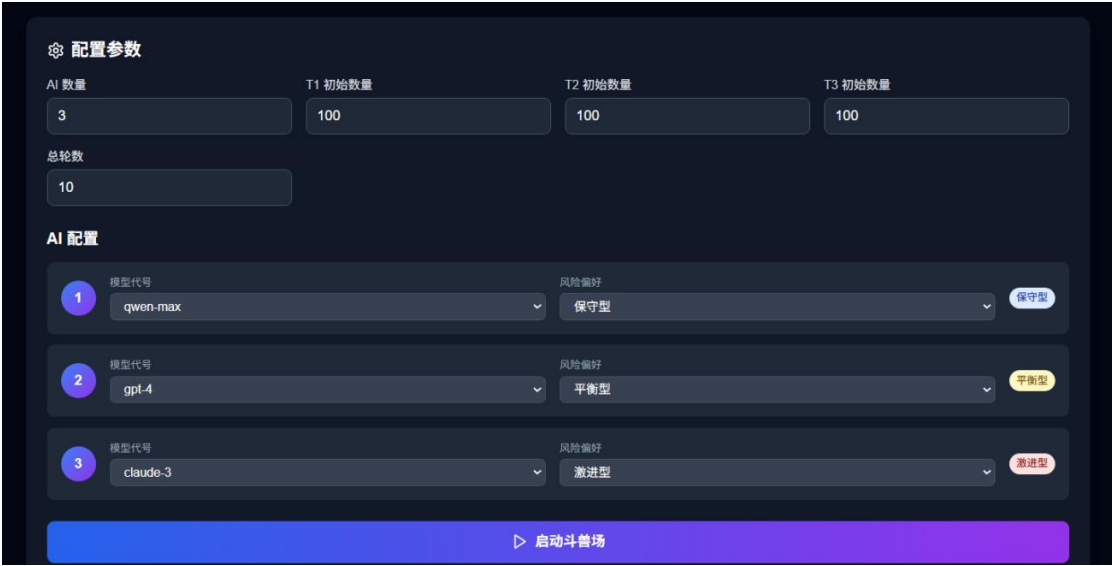


图 6.11 斗兽场配置页面

配置完成后点击启动，系统初始化三位 AI 的链上地址，向各地址转入初始代币，并在前端跳转至监控页面。

6.3.2 博弈过程与资产变化

本次测试中，斗兽场共运行 10 轮。第一轮中，三位 AI 并发读取市场数据后按思考速率排序，依次完成代币兑换决策。图 6-13 展示了 10 轮博弈中三位 AI 的资产总值（以 SET 折算）变化曲线。



图 6.13 斗兽场资产总值变化曲线

6.3.3 决策日志与链上记录

每轮博弈结束后，系统记录 AI 的决策理由与操作内容，并在前端日志区域展示。同时，测试链上发生的每笔转账均被记录。



图 6.14 斗兽场决策日志

图 6.15 展示的是链上交易记录，包含了每次交易从 Agent 地址流向管理员地址，再从管理员地址流回 Agent 地址的交易日志。



图 6.15 链上交易记录

Agent 在观察到前两位 Agent 操作引发的价格变动后，基于保守策略选择了轻仓跟入，这验证了二次决策机制的有效性——后序 Agent 能够对前序操作引发的市场变化做出反应。

6.4 本章小结

本章对系统的两个核心功能模块进行了测试与展示。股票辅助交易场景中，不同风险偏好的 AI 在相同市场条件下给出了差异化的决策，AI 对市场的分析和风险评估能够对用户起到一定的参考作用；区块链斗兽场中，三位 AI 在 10 轮博弈中呈现出与各自偏好相符的资产变化曲线。整体而言，系统各模块运行稳定，达到了预期的设计目标。

7 总结与展望

7.1 本文工作总结

本课题构建了一个面向大模型交易行为模拟的复合系统：本系统搭建了真实股票行情与链上代币博弈两种场景，实现了多 Agent 的并发决策、工具调用与资产结算的完整闭环。其核心工作可概括为以下三点：

其一，深度集成 LangChain4j 框架与 SpringBoot 脚手架，以 @Tool 注解将 Java 方法提供给 Agent 使用，并用 AiServices 代理封装各种模型，使不同厂家的大模型能够在 Java 生态下调用统一的工具接口并行运作。

其二，笔者设计了一套链下定价、链上交易的代币博弈机制。三种 ERC-20 代币的兑换价格由线性冲击公式驱动，该设计将 GasFee、流动性深度等复杂变量从系统中剥离，使得笔者不用实现一整个 uniswap 交易所就能实现链上交易功能，也能够让用户更专注于 Agent 的推理过程。

其三，在斗兽场模块中引入了基于思考速率竞争机制。三位代理并发读取数据后，按思考速率排序依次执行交易，并且在二次决策中，后序者可观察到前序操作引发的价格变动。这一设计模拟了真实的交易情况，让程序一次仅执行一笔交易，这既解决了最复杂的线程安全问题，同时也让 AI 之间产生了博弈。

7.2 不足与改进方向

现在的系统仍然存在着几项实质性的局限，这些局限点都会影响模拟交易的表现与可推广性。

行情数据的局限。Alpha Vantage 的免费 API 存在限额的情况，无法支撑真实场景下的高频调用，且股票列表配置在本地，不支持动态扩展。后续笔者会调研更多数据源厂商或按需购买 API 资源，并且将本地配置迭代至线上的配置中心，实现在线扩展股票列表和其他配置。

简单的定价模型。真实的去中心化交易所使用恒定乘积公式进行定价，该计算方式会产生非线性的滑点，而本系统使用的线性冲击公式则忽略了真实交易中的非线性滑点，也未引入 Gas 竞拍、区块确认延迟等链上细节。改进方向是将定价逻辑迁移至链上，实现类似 uniswap 的交易所功能，使用自动做市商对 Token 定价，或者使用链上真实的历史价格数据，使定价机制更贴近实际市场的微观情况。

工具集较为单一。当前 Agent 仅能够通过观测 k 线图，调用最简单的 buy/sell 功能，而在真实市场中，往往有很多数据之外的信息能够成为交易决策的重要参考，这些信息同样也是 Agent 重要的博弈因素。所以笔者后续计划引入更多种类

的工具，比如：新闻查询工具、情绪分析工具等等，并且对 Agent 在信息权限和工具集上进行差异化配置，以呈现出工具和信息差对决策的关键作用。

试验功能不足。评价指标单一，仅以资产作为评价参考，没有考虑到夏普比率、最大回撤等风险调整后指标；试验操作繁琐，不支持同一配置下的重复试验和统计，回归测试需要手动记录结果。后续计划调研更多绩效指标，集成多维度评价体系，并在实际使用后，对操作交互进行迭代，适配更多场景下的试验需求。

上述不足指明了本系统从“可用”向“可靠”演进的努力方向。后续若能继续迭代，本系统有望为大语言模型运用在金融场景的最佳实践。

参 考 文 献

- [1] Hedge funds battle to turn ChatGPT from intern to analyst[N/OL]. Bloomberg, 2024 [2026-05-04]. ndtvprofit.com.
- [2] EVANS B. Bridgewater starts \$2 billion fund that uses machine learning for decision-making and will include models from OpenAI, Anthropic and Perplexity[N/OL]. Fortune, 2024-07-01 [2026-05-04]. fortune.com.
- [3] SON H. JPMorgan is developing a ChatGPT-like AI service that gives investment advice [N/OL]. CNBC, 2023-05-25 [2026-05-04]. cnbc.com.
- [4] BLOCKWORKS. The 5 biggest DeFi hacks of 2023[EB/OL]. (2023-12-22) [2026-05-04]. blockworks.co.
- [5] YANG H, LIU X Y, WANG C D. FinGPT: open-source financial large language models [EB/OL]. arXiv:2306.06031, 2023 [2026-05-04]. arxiv.org.
- [6] LI Y, YU Y, LI H, et al. TradingGPT: multi-agent system with layered memory and distinct characters for enhanced financial trading performance[EB/OL]. arXiv:2309.03736, 2023 [2026-05-04]. arxiv.org.
- [7] JAVARE P, ROY S. SlipSwap: reduce the slippage that is incurred during the swap of tokens using algorithmic analysis[C]//2021 23rd ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP). Rotterdam: IEEE, 2021.
- [8] STRUCHKOV I, LUKASHIN A, KUZNETSOV B, et al. Agent-based modeling of block chain decentralized financial protocols[C]//29th Conference of Open Innovations Association (FRUCT 2021). Tampere, Finland: IEEE, 2021: 337-343. DOI:10.23919/FRUCT52173.2021.9435601.
- [9] ZHANG C, LIU X Y, JIN M Y, et al. When AI meets finance (StockAgent): large language model-based stock trading in simulated real-world environments[EB/OL]. (2024-07-15) [2026-05-04]. arxiv.org.
- [10] 何杉杉, 周雅兰, 郭宇阳. 基于 LSTM 和 DDPG 的股票交易决策算法[J/OL]. 南京大学学报(自然科学), 2024, 60(6): 940-953 [2026-05-04]. DOI:10.13232/j.cnki.jnju.2024.06.006.
- [11] ERNIE 5.0 Technical Report[J]. 2026.
- [12] Qiu Z , Wang Z , Zheng B ,et al.Gated Attention for Large Language Models: Non-linearity, Sparsity, and Attention-Sink-Free[J]. 2025.
- [13] OpenAI. GPT-4o mini model documentation[EB/OL]. [2026-05-04]. openai.com.
- [14] Pivotal Software. Spring Boot documentation[EB/OL]. [2026-05-04]. spring.io.

- [15] YOU Y X. Vue 3 documentation[EB/OL]. [2026-05-04]. vuejs.org.
- [16] Alpha Vantage Inc. Alpha Vantage API documentation[EB/OL]. [2026-05-04]. alphavantage.co.
- [17] LangChain4j. LangChain4j documentation[EB/OL]. [2026-05-04]. langchain4j.dev.
- [18] VOGELSTELLER F, BUTERIN V. ERC-20: token standard[EB/OL]. Ethereum Improvement Proposals, no. 20, 2015-11-19 [2026-05-04]. ethereum.org.
- [19] Web3j. Web3j documentation[EB/OL]. [2026-05-04]. web3j.io.
- [20] OpenZeppelin. ERC20: OpenZeppelin contracts API reference[EB/OL]. [2026-05-15]. openzeppelin.com.
- [21] Truffle Suite. Ganache documentation[EB/OL]. [2026-05-04]. trufflesuite.com.
- [22] The Apache Software Foundation. Apache Commons Math user guide[EB/OL]. [2026-05-04]. apache.org.

致 谢

时光荏苒，大学四年的学习即将落下帷幕。本毕业设计的完成，不仅是对我大学四年所学的总结，更离不开在此期间给予我帮助和 supports 的老师、同学和家人。

首先我要真挚感谢我的指导老师桑军，在论文的选题与撰写过程中，桑军老师给予了宏观方向上的建议。其严谨的学术作风和对专业领域的深刻理解，为我开展系统研发工作提供了重要的参考。在此，向老师表达诚挚的敬意。

其次，我要感谢我的同学们，他们在我专业知识学习和就业方向上起到了非常大的帮助，我们共同探讨技术难题，大家的建议帮助我克服工作中的困难。

最后我要向我的家人表达最深切的谢意。感谢你们从我幼时起就在学业上无条件地支持与理解我，让我能够全身心地投入到学习中；感谢你们在我本科期间，支持我选择的职业道路，相信我的决定。

由于本人水平有限，论文中难免存在疏漏之处，恳请各位评审老师批评指正。

原创性声明

郑重声明：所呈交的论文（设计）《基于大模型和区块链的加密货币模拟交易所的设计与实现》，是本人在导师的指导下，独立进行研究取得的成果。除论文（设计）中已经标注引用的内容外，本论文（设计）不包含其他人或集体已经发表或撰写过的作品成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律后果，并承诺因本声明而产生的法律结果由本人承担。

论文（设计）作者签名： 胡力文

日期： _____

使用授权书

本论文（设计）作者完全了解学校有关保留、使用论文（设计）的规定，同意学校保留并向国家有关部门或机构送交论文（设计）复印件和电子版，允许论文（设计）被查阅和借阅。本人授权重庆大学将本论文（设计）的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制方式保存和汇编本论文（设计）。

本论文（设计）属于：

保 密 ☐ 在____年解密后适用本授权书

不保密 ☒

论文（设计）作者签名： 胡力文 指导教师签名： _____

日期： _____

日期： _____